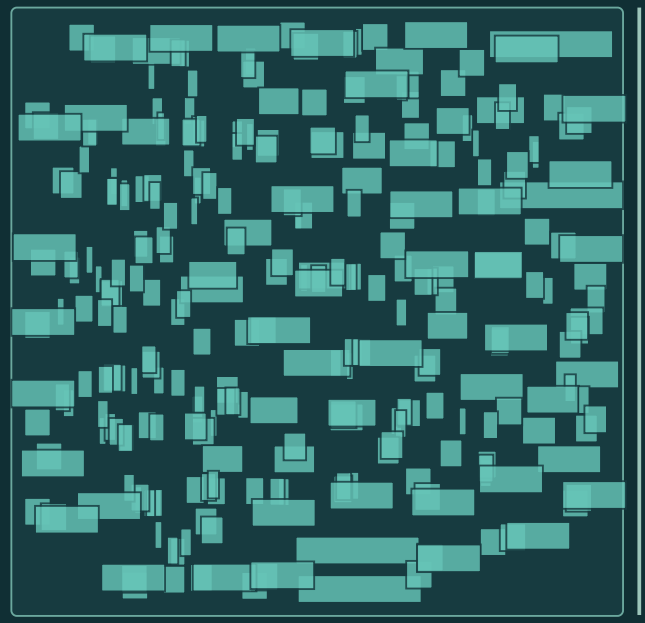
GPL：从连接到空间

把数十万单元的位置，
写成一个可以迭代求解的问题。

中文深度导读 · 56 页
建议：入门 25 分钟 / 完整分享 90–120 分钟

源码基线：zint99/OpenROAD · 977813f3b02e
图例：青色 = 单元 / 线长；橙色 = 密度 / 拥塞

GCD / 294 CELLS / NANGATE45



30.8 μm

已有回归布局重绘 · simple01.defok · 尚未 DPL

> 理解三个动作：拉近连接、疏散密度、引入物理反馈。

来源：simple01.defok · 已有布局 ↗ · GPL 模块说明 ↗

READING MAP · 按重要性分配时间

两条阅读路径，同一条算法主线

快速入门 · **25** 分钟

01–07 → 10 → 15、18、20–21 → 28、32、34 → 35、39、42 → 50、56

算法深读 · **90–120** 分钟

顺序阅读 08–44；重点推导 WA、Poisson 与 Nesterov，并操作四个实验。

开发者延伸

45–55：数据边界、源码导航、诊断与回归设计。

01 直觉与建模
01—07

02 线长 / 密度 / Nesterov
08—34

03 时序 / 可布线性
35—44

04 源码 / 诊断 / 练习
45—56

原理示意 · 非实测运行

> 核心算法占 28 页；时序与拥塞反馈占 10 页。

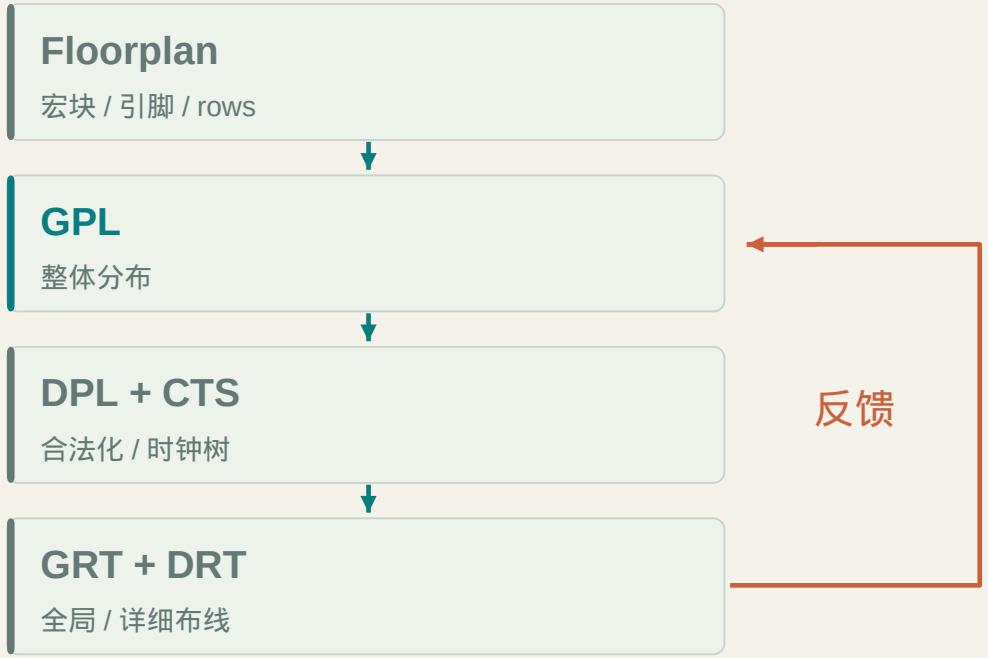
来源：GPL 模块说明 ↗

GPL 决定整体分布，DPL 完成合法落位

输入：网表、单元尺寸、core / rows、固定对象、I/O 与区域约束。

输出：可移动实例的坐标，以及布局质量指标。

时序驱动可保留 resizer 修改，因此部分运行也会改变网表和单元面积。



原理示意 · 非实测运行

> 全局布局允许暂时重叠；最终还需要合法化与布线验证。

来源：GPL 模块说明 ↗ · 时序反馈编排 ↗

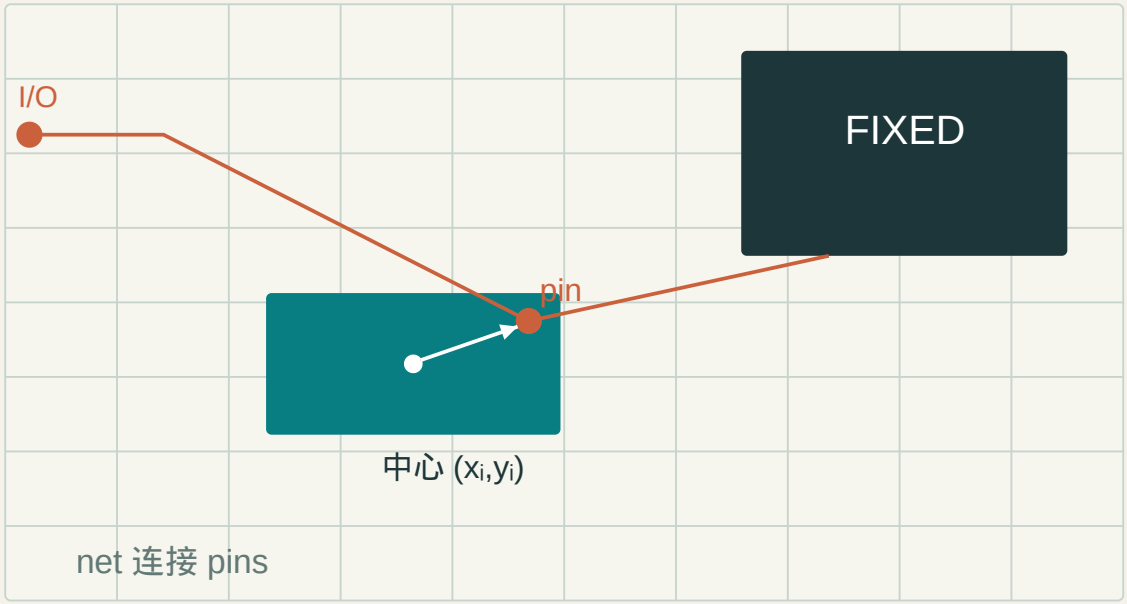
优化变量是单元中心，连接端点是 pin

Movable cell：中心坐标 (x_i, y_i) 可变。

Fixed macro / blockage：占空间但不参与移动。

Pin：位置 = cell 中心 + 变换后的 offset；I/O 可提供固定端点。

Net：连接多个 pin 的超边，不能只看 cell 中心距离。



原理示意 · 非实测运行

> 尺寸、坐标、pin offset 和可移动状态，共同决定正确的力。

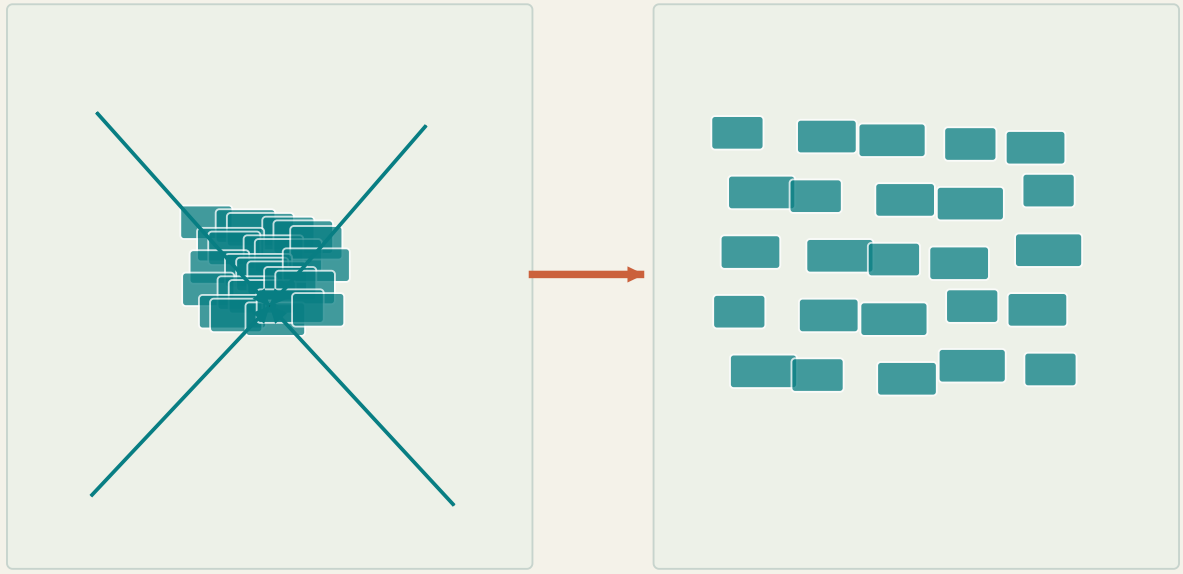
来源：PlacerBase · 数据建模 ↗ · GCell / GPin / GNet ↗

只追求短线长，会把电路拉成一团

线长力：连接越近越好。

密度力：为每个单元分配足够空间。

均匀不是目的本身：还要兼顾关键路径、宏块通道和可布线性。



线长优先 → 聚集　　加入密度 → 展开

原理示意 · 非实测运行

> 布局的难点，是在连接结构和空间容量之间持续寻找平衡。

来源：WA 线长与力 ↗ · 密度力回采样 ↗

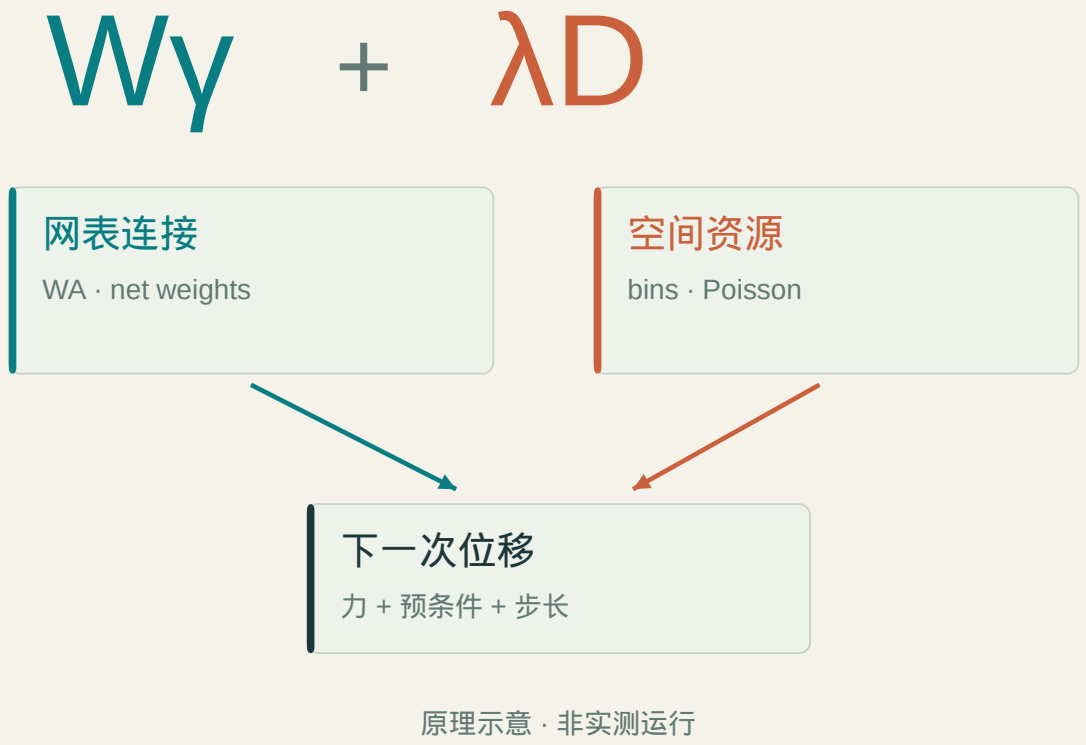
把位置优化写成两个主要代价

$$\min_{z \in \Omega} \quad W_\gamma(z;\, \omega) + \lambda\, D(z)$$

z：所有可移动中心；Ω：允许移动的区域。

W_γ：平滑线长，ω 是 net 权重。

D：静电密度能量；λ 平衡线长与疏散强度。



原理示意 · 非实测运行

γ、λ、net 权重与虚拟面积会变化：整个运行不是固定目标的单次下降。

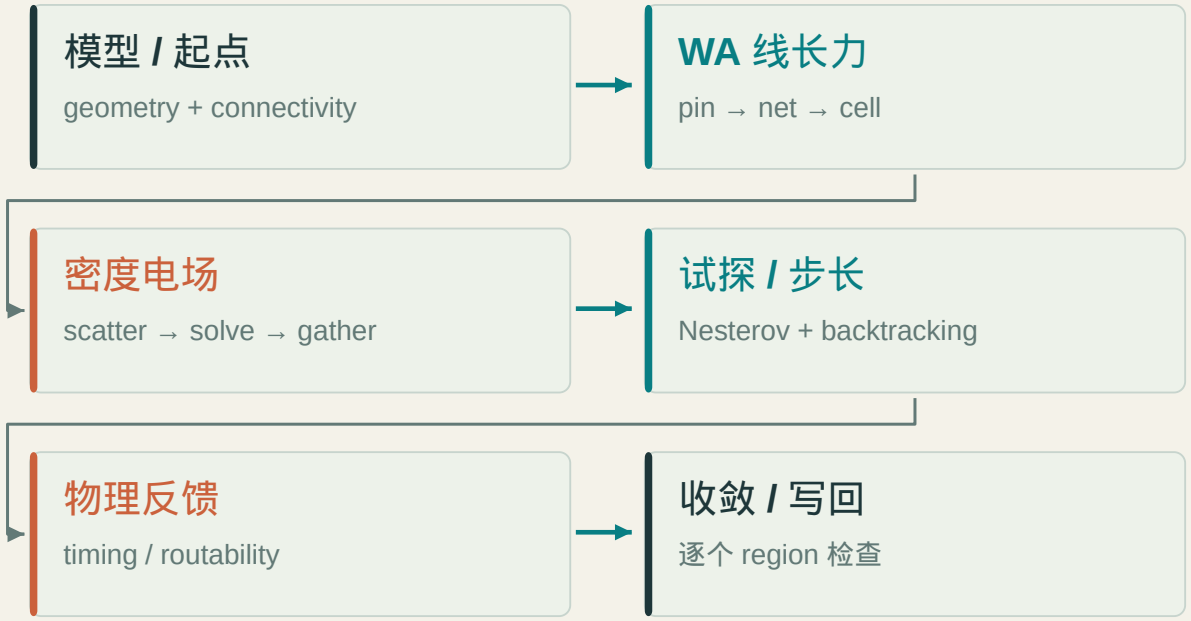
来源：力合成与预条件 ↗ · Nesterov 主循环 ↗

完整算法是一条内循环，外加两种反馈

起点：建立布局模型；可选 B2B 初始布局。

高频循环：算线长力与密度场 → 更新位置 → 检查步长。

低频反馈：在合适阶段评估时序和拥塞，再修改模型继续迭代。



原理示意 · 非实测运行

> Initial placement 提供起点；Nesterov placement 负责主要优化。

来源：Nesterov 主循环 ↗ · InitialPlace · B2B 建模 ↗

初始布局：用 B2B 弹簧得到有结构的起点

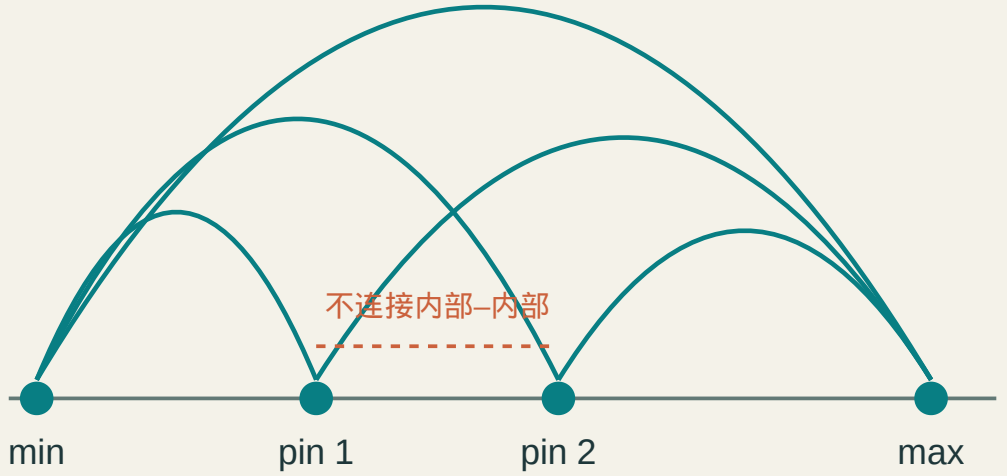
每个方向分别找 net 的极值 pin，把边界端点与其他端点连接。

$$w_{ij} \propto 1 / \max(|x_i - x_j|, \varepsilon)$$

固定端点进入右端向量；可移动端点进入稀疏矩阵。

更新 pin 极值和权重，再解 x / y 两个系统。

一个方向：只保留与极值端点相连的边



原理示意 · 非实测运行

> 稀疏 B2B 模型降低连接复杂度，但此时还没有解决重叠。

来源：InitialPlace · B2B 建模 ↗ · BiCGSTAB 求解器 ↗

两单元手算：固定端点如何确定位置

$$E = \tfrac{1}{2}[x_1^2 + (x_1 - x_2)^2 + (x_2 - 10)^2]$$

对 x_1、x_2 求导并令其为零：

$$2x_1 - x_2 = 0$$

$$-x_1 + 2x_2 = 10$$

$x_1 = 10/3$，$x_2 = 20/3$。 两条方程残差均为 0。



矩阵 A

[2 −1 ; −1 2]

右端 b

[0 ; 10]

$$Ax = b \rightarrow \nabla E = 0$$

原理示意 · 非实测运行

有锚点的连接决定相对位置；没有面积约束，单元仍可能重叠。

来源：InitialPlace · B2B 建模 ↗ · BiCGSTAB 求解器 ↗

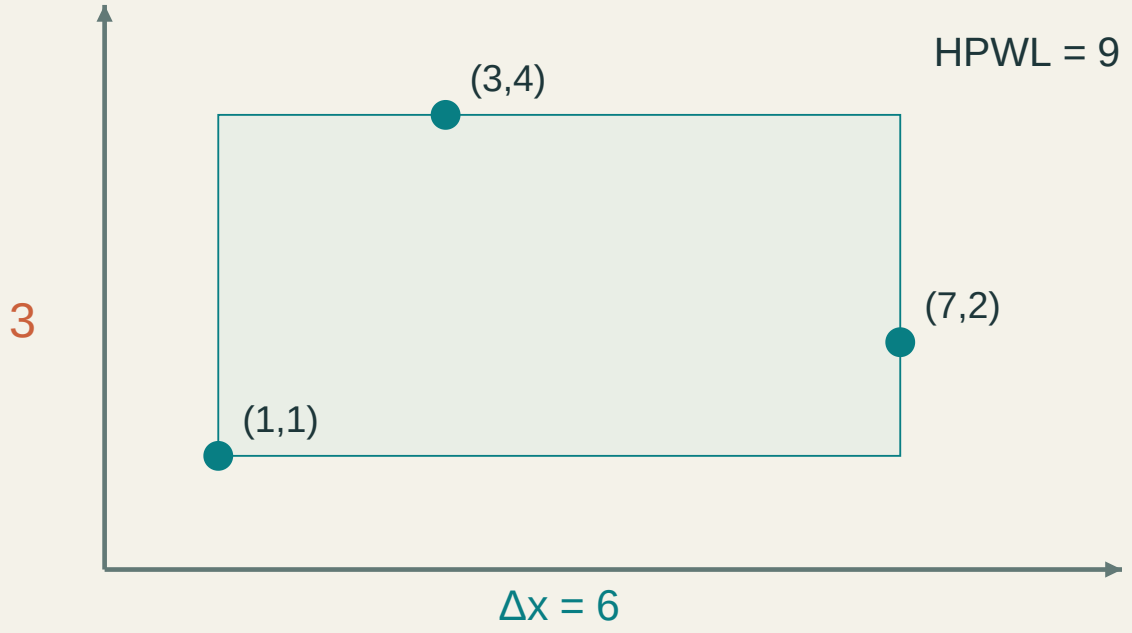
HPWL：用包围盒估计一条 net 的线长

$$HPWL_e = (x_{max} - x_{min}) + (y_{max} - y_{min})$$

示例 pin：(1,1)、(3,4)、(7,2)。

宽 6 + 高 3 = **9** 个长度单位。

不含绕障、层分配、via 与详细布线拓扑。



原理示意 · 非实测运行

> HPWL 适合快速比较，但单独优化它会遇到不可导与梯度稀疏。

来源：WA 线长与力 ↗ · GPL 模块说明 ↗

WA：把 max / min 换成平滑的加权平均

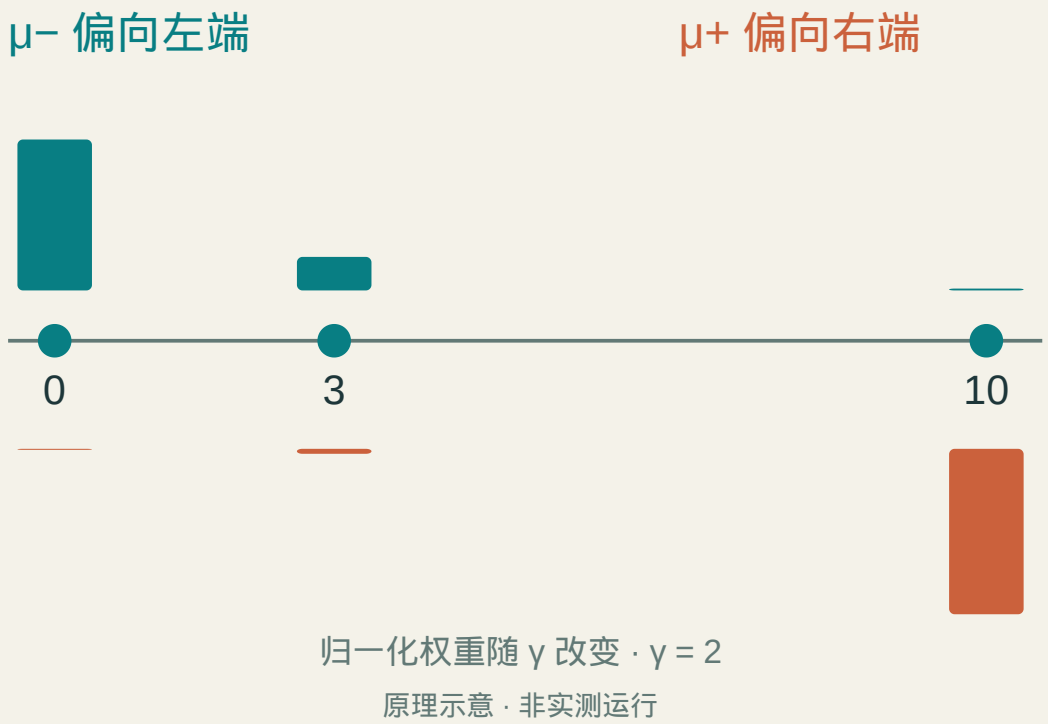
$$\mu^{+} = \Sigma\, x_i e^{x_i/\gamma} \,/\, \Sigma\, e^{x_i/\gamma}$$

$$\mu^{-} = \Sigma\, x_i e^{-x_i/\gamma} \,/\, \Sigma\, e^{-x_i/\gamma}$$

$$W_{\gamma,x} = \mu^{+} - \mu^{-}$$

γ 大：更平滑，极值选择更柔和。

γ → 0^{+}：μ^{+} → max x，μ^{-} → min x。



> WA 不是 log-sum-exp；这里使用的是指数加权的坐标平均。

来源：WA 线长与力 ↗

对 WA 求导，得到每个 pin 的拉力

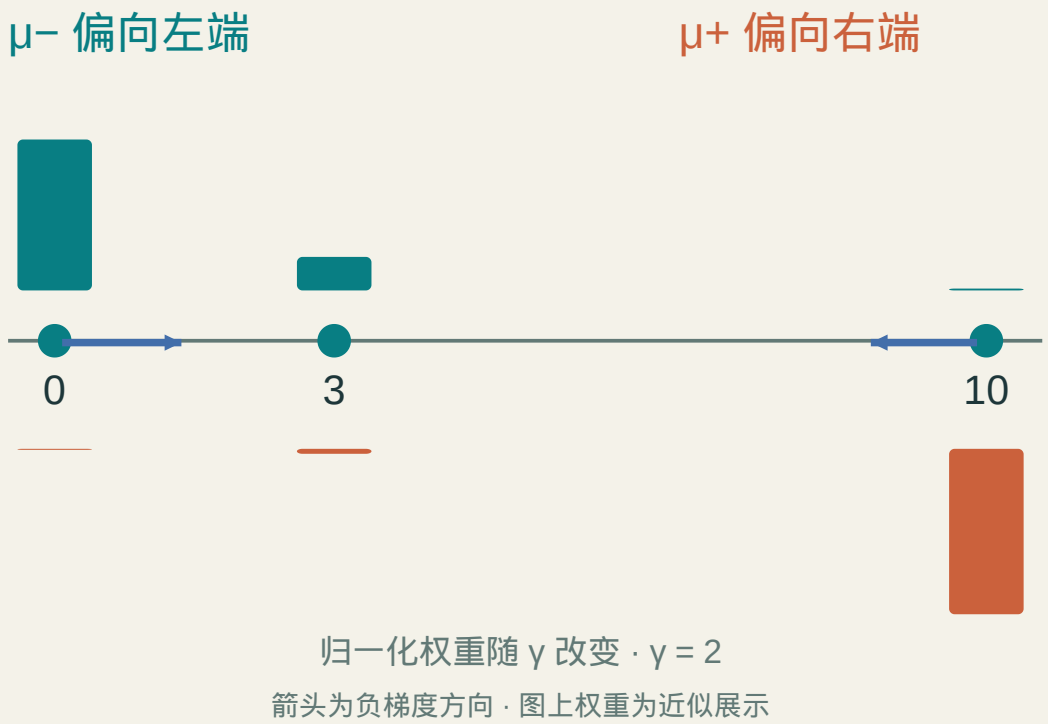
$$p_i^+ = e^{x_i/\gamma} / \Sigma e^{x_j/\gamma}$$

$$\partial \mu^+/\partial x_i = p_i^+[1 + (x_i - \mu^+)/\gamma]$$

$$\partial \mu^-/\partial x_i = p_i^-[1 - (x_i - \mu^-)/\gamma]$$

数学梯度：$\partial W/\partial x_i = \partial \mu^+/\partial x_i - \partial \mu^-/\partial x_i$。

源码返回 **min 导数 − max 导数**，即负梯度方向。



归一化权重随 γ 改变 · γ = 2

箭头为负梯度方向 · 图上权重为近似展示

从 pin 汇总到 cell，并乘 net 权重；固定 pin 参与算力，但不被移动。

来源：WA 导数与符号 ↗

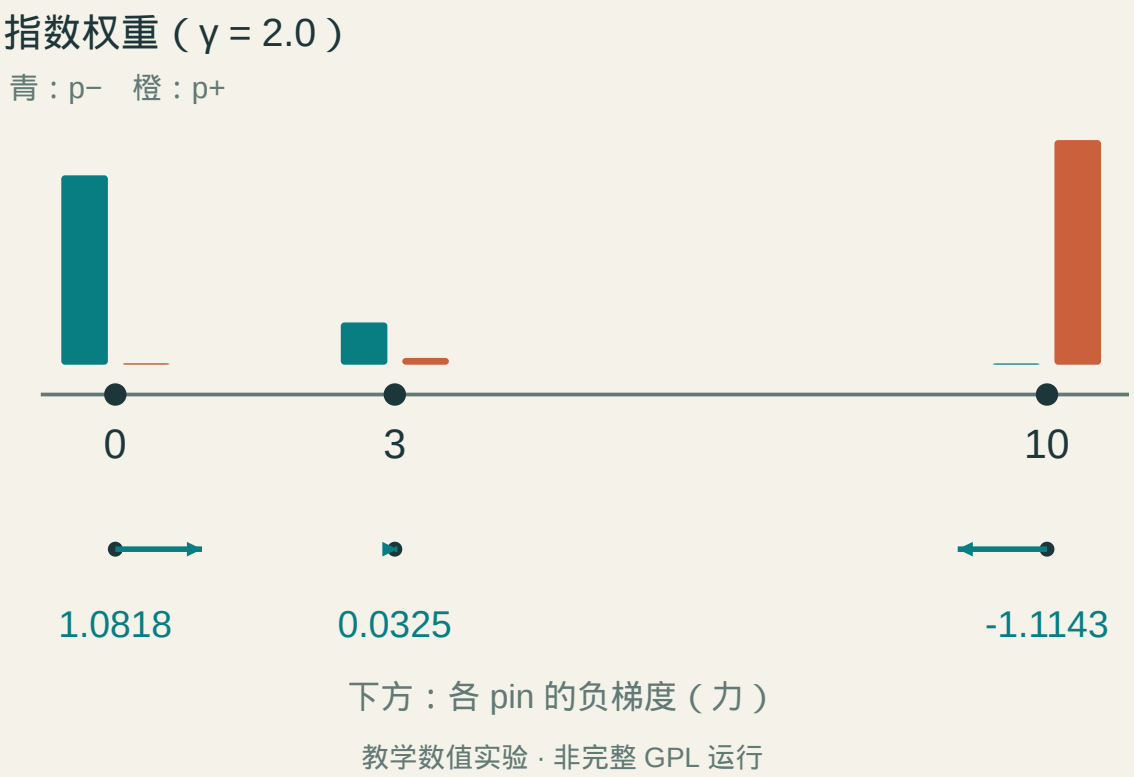
实验 ①：调 γ，观察近似值和力如何变化

固定一维 pin：**0**、**3**、**10**。

拖动 γ，比较 HPWL = 10 与 WA；箭头显示各 pin 的负梯度。

小 γ 更接近跨度；大 γ 更平滑，但不等于更强的力。

WA = 9.1321 | HPWL = 10



> 实验实时求值与求导；不是完整 GPL，也不是实测布局曲线。

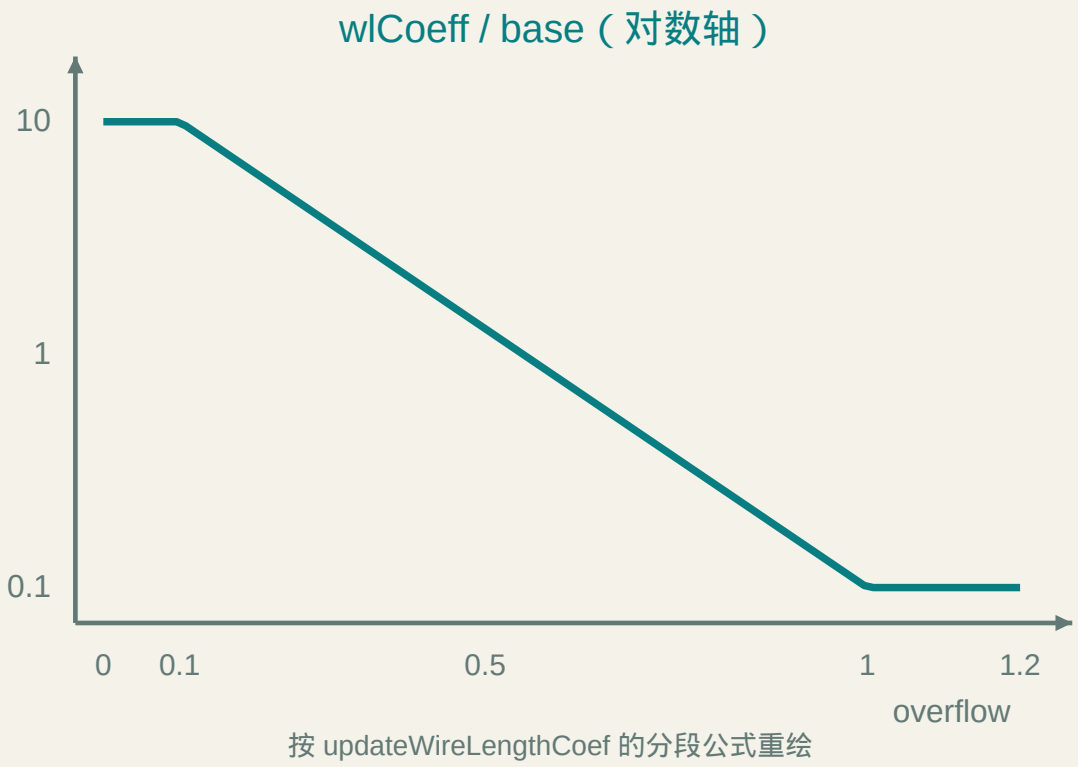
来源：WA 线长与力 ↗ · WA 导数与符号 ↗

稳定指数计算，并随 overflow 收紧近似

用 $e^{(x_i - x_{max})/\gamma}$ 和 $e^{(x_{min} - x_i)/\gamma}$，把指数约束为非正。

减去共同常数不改变归一化加权平均。

源码随 overflow 下降增大 wlCoeff，即减小等效 γ，让线长模型更接近 HPWL。



按 updateWireLengthCoef 的分段公式重绘

> 从平滑到尖锐：早期便于移动，后期提高线长代理的分辨率。

来源：WA 线长与力 ↗ · 平滑系数调度 ↗

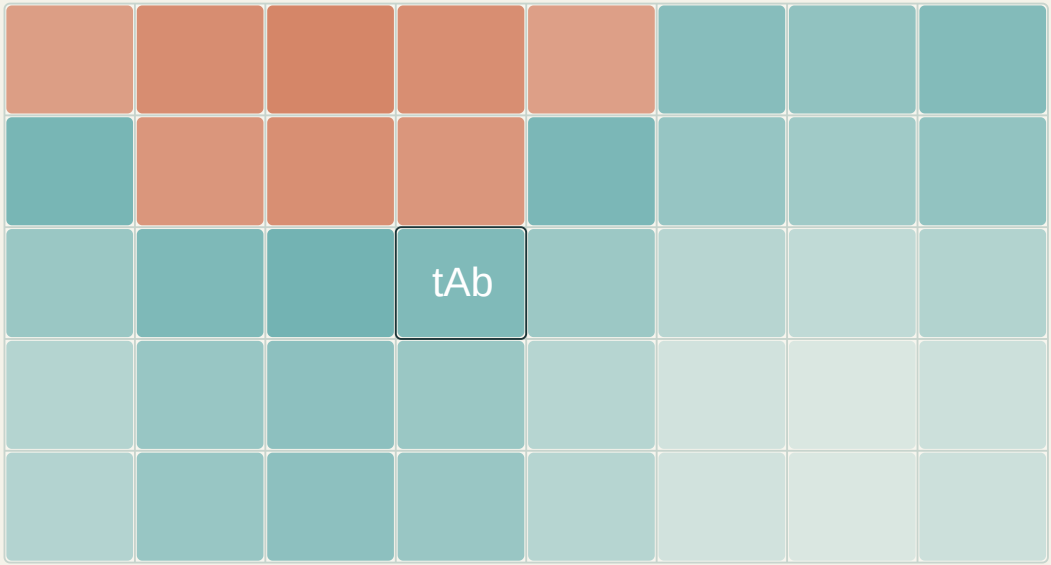
把连续版图切成 bins，才能计算局部容量

每个 bin 有面积 A_b 和目标密度 t_b。

容量 $C_b = t_b A_b$

一个大单元跨越多个 bin，按重叠面积分摊占用。

网格太粗会掩盖热点；太细会增加计算量与数值敏感性。



深色 = 较高占用 · 每格分别计算容量

原理示意 · 非实测运行

> 要衡量“这个地方被占了多少面积”，不能只数中心点。

来源：面积散射与 overflow ↗ · Filler 初始化 ↗

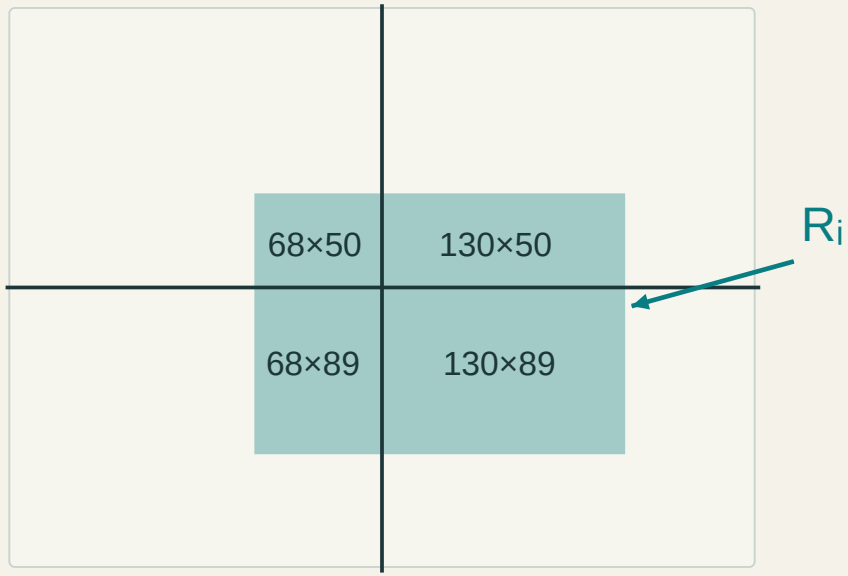
Scatter：一个 cell 的面积分摊到邻近 bins

$$a_i b = \text{overlap}(R_i, B_b) \times s_i$$

R_i 是密度矩形；s_i 是面积缩放系数。

遍历与 R_i 相交的 bin，累加实际重叠面积。

位置移动时，多个 bin 的占用连续交换。



Σ overlap = 198 × 139

原理示意 · 非实测运行

> 同一个几何重叠算子，既用于密度散射，也用于电场回采样。

来源：面积散射与 overflow ↗ · 密度力回采样 ↗

密度平滑：扩大支撑范围，但守恒总面积

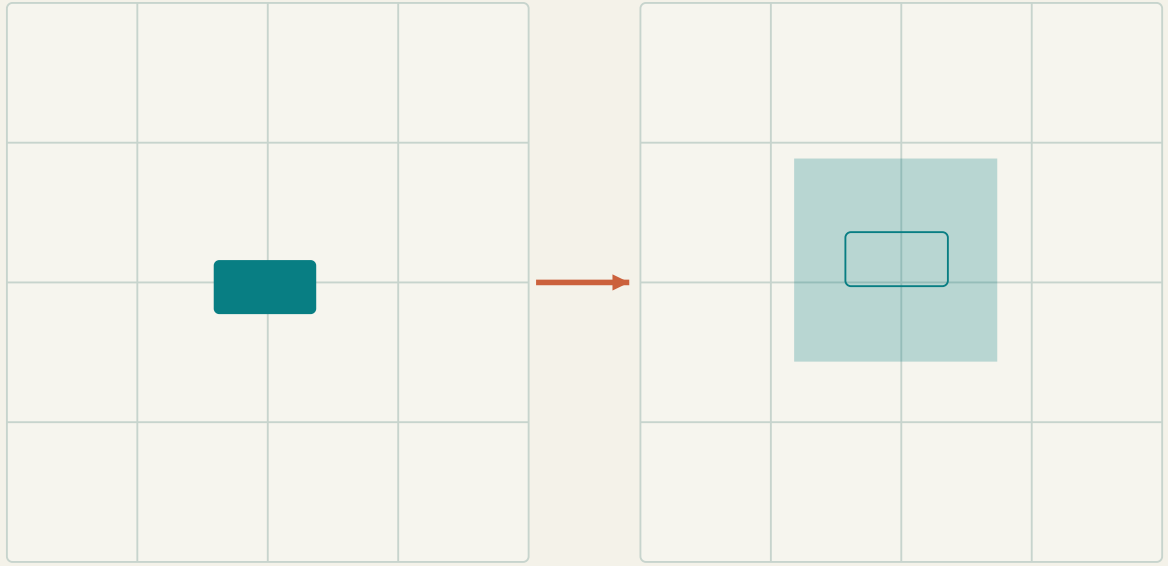
$$d_x = \max(w, \sqrt{2}\cdot b_x)$$

$$d_y = \max(h, \sqrt{2}\cdot b_y)$$

$$s = wh / (d_x d_y)$$

矩形变宽，单位面积贡献相应变小。

$s\cdot d_x d_y = wh$：总“电荷”保持一致。



原理示意 · 非实测运行

> 密度平滑守恒面积；拥塞膨胀会增加虚拟面积，二者不同。

来源：密度尺寸与缩放 ↗

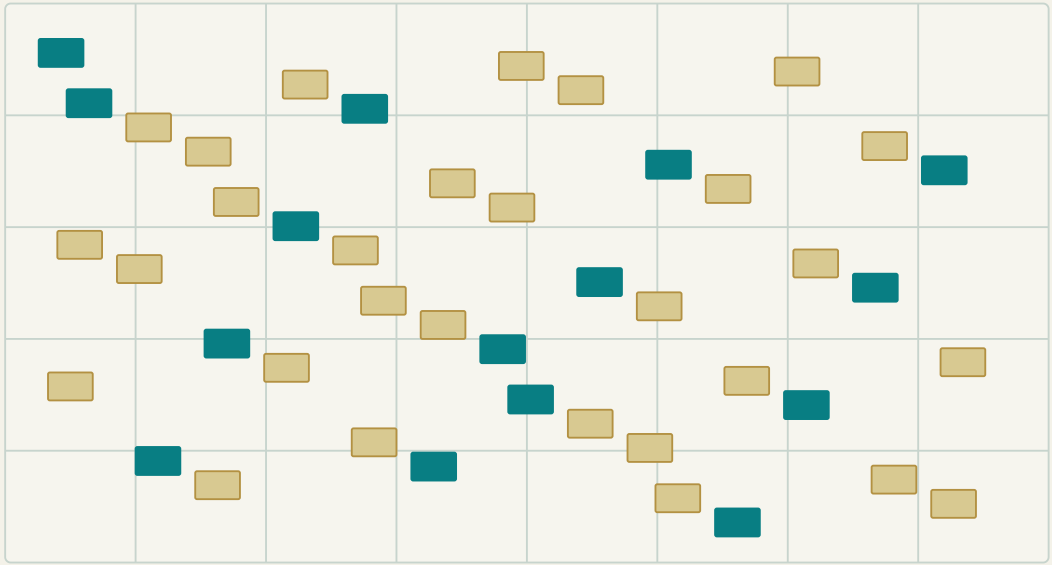
Filler：用虚拟电荷表示未被真实单元占用的空间

$$A_{filler} \approx t \cdot A_{free} - A_{inst}$$

无 net 连接，只响应密度力。

填补目标容量与实例面积的差额，让密度场有合理的平衡状态。

它们不是后续制造流程中插入的物理 filler cells。



原理示意 · 非实测运行

> Filler 参与密度场；报告真实拥挤程度时，overflow 分子排除 filler。

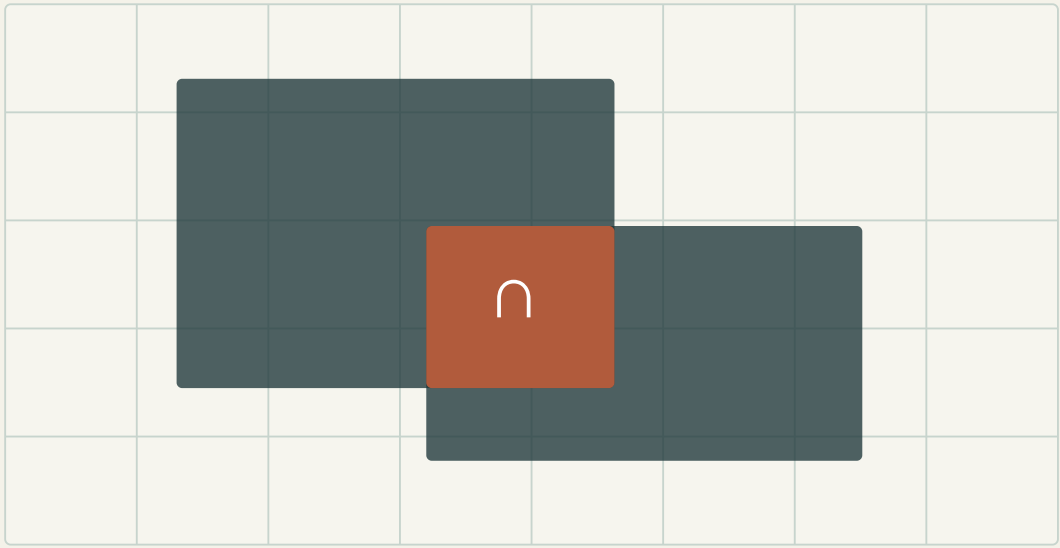
来源：Filler 初始化 ↗ · 面积散射与 overflow ↗

固定宏块与障碍：没有自由空间，就不能分配容量

固定对象变成不可放置面积，进入 bin 的占用背景。

重叠的宏块 / blockage 必须按几何并集统计，避免重复扣减。

当前实现对固定背景使用 target density 缩放，并区分 scaled / unscaled 路径。



$$A \cup B = A + B - A \cap B$$

交叠区域只计一次 · 固定背景按实现进行密度缩放

> 宏块移动状态、几何可用区域和密度归一化必须一起理解。

来源：固定面积并集与缩放 ↗ · 面积散射与 overflow ↗

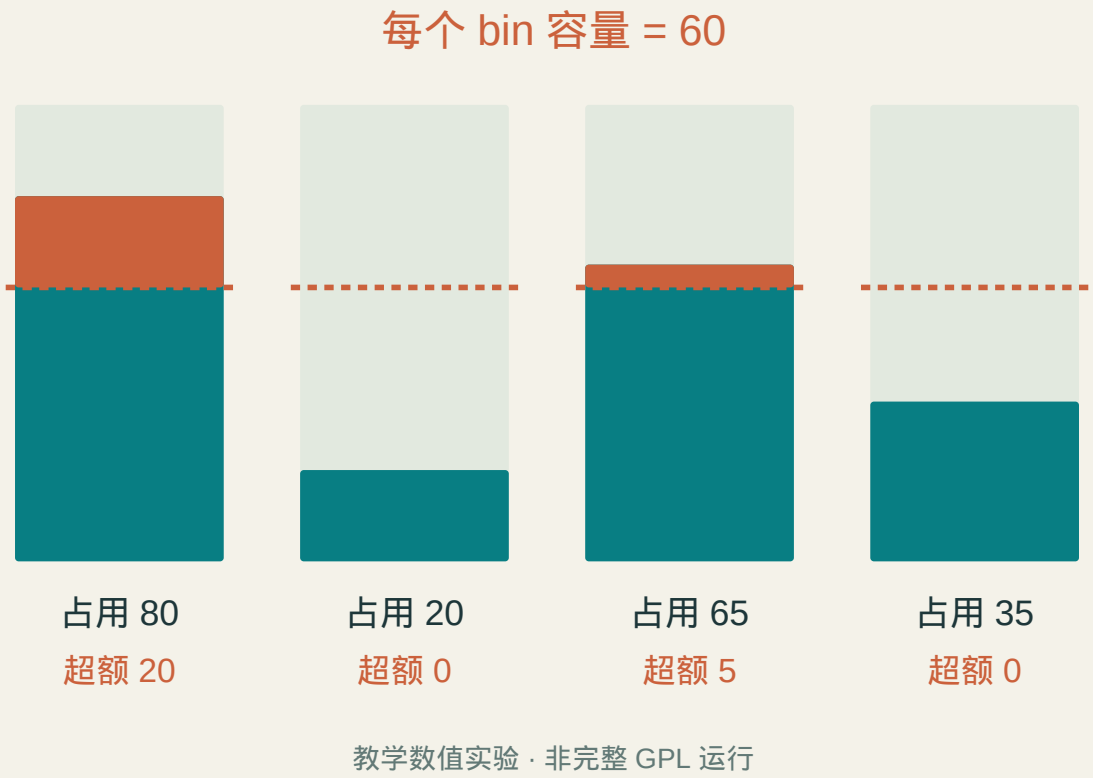
实验 ②：同一组占用，目标密度改变 overflow

实例占用：**[80, 20, 65, 35]**，总面积 200；无宏块、无 filler、无障碍。

$$O = \Sigma \max(0, a_b - 100t) / 200$$

超额面积 = 25.0 | O = 0.1250

这是标准单元简化算例；真实代码还有宏块与归一化保护。



教学数值实验 · 非完整 GPL 运行

> t = 0.60 时：超额面积 20 + 0 + 5 + 0 = 25，overflow = 0.125。

来源：面积散射与 overflow ↗ · 迭代 overflow 分母 ↗

静电类比：让密集区域产生向外的力

$$-\nabla^2\psi = \rho - \bar{\rho}$$

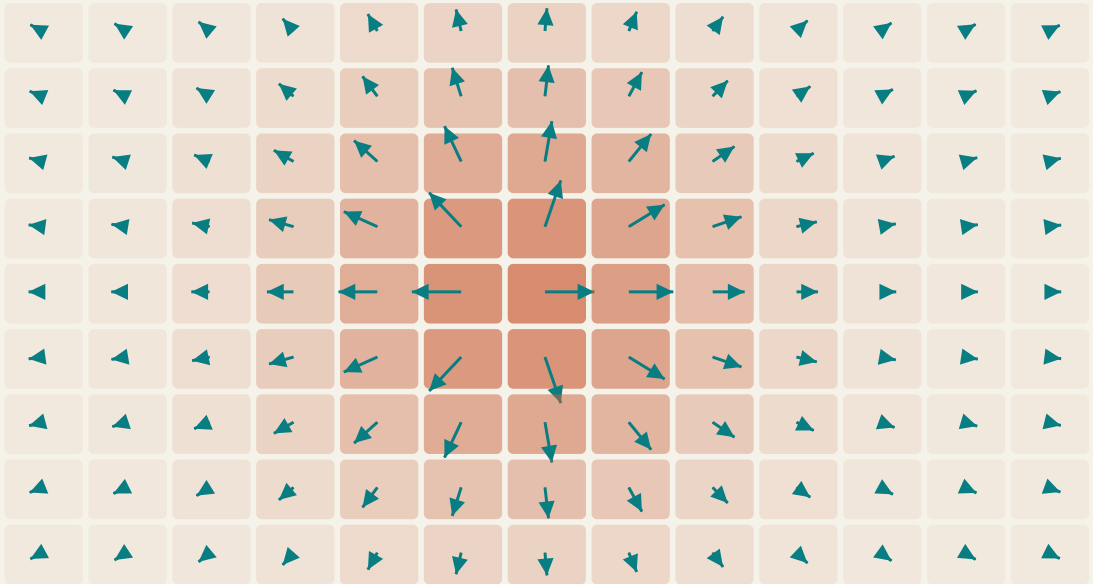
$$D \propto \tfrac{1}{2}\int (\rho-\bar{\rho})\psi \, dA$$

$$E = -\nabla\psi$$

ρ：归一化密度；$\bar{\rho}$：空间平均。

ψ：电势；D：静电能量；E：电场。

高密区域对远处也有影响，不只推动相邻 bin。



中心较密 → 周围电场引导疏散

原理示意 · 非实测运行

> 电场来自整个密度分布；并非把每个热点独立地推向最近空格。

来源：CPU DCT Poisson 求解 ↗ · 密度力回采样 ↗

在频域求 Poisson：微分变成乘法

$$\hat{\psi}_{uv} = \hat{\rho}_{uv} / (k_u^2 + k_v^2)$$

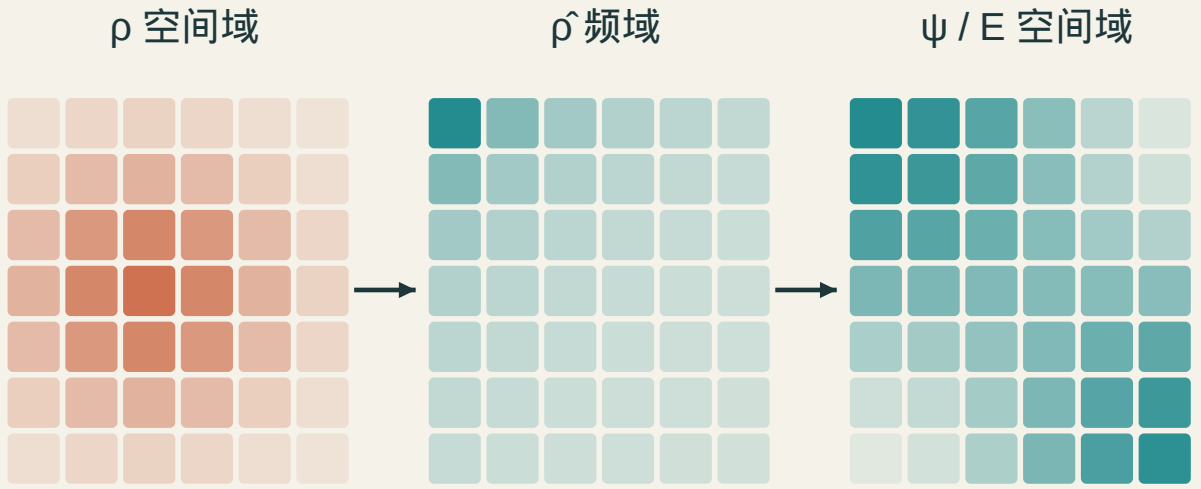
$$\hat{\psi}_{00} = 0$$

① 密度网格做 DCT。

② 每个非零频率除以波数平方。

③ 逆变换得到电势与两个电场分量。

规则网格上的变换通常为 O(B log B)，B 是 bin 数。



非零频率：除以 $kx^2 + ky^2$

(0,0) 模式清零；方向尺度与逆变换归一化相配

原理示意 · 非实测运行

零频必须单独处理；非方形 bin 的两个方向尺度也不能混用。

来源：CPU DCT Poisson 求解 ↗

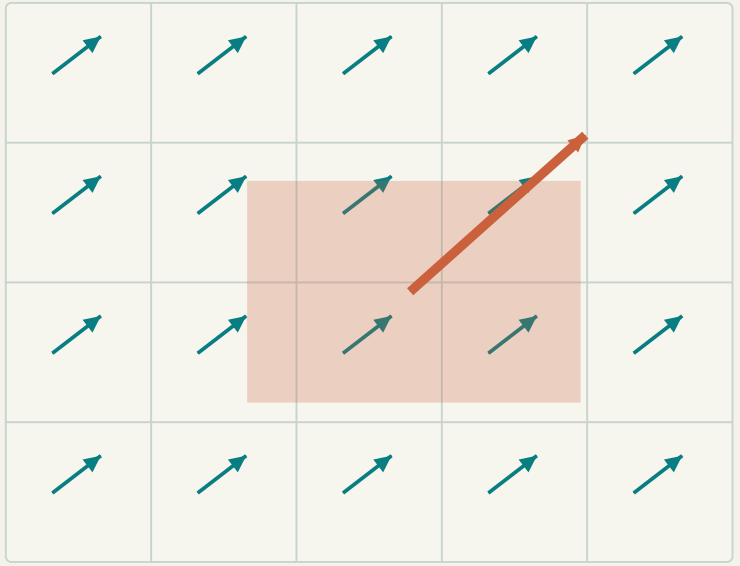
Gather：把 bin 电场积分回每个 cell

$$F_{D,i} \approx \Sigma_b \, a_i b \, E_b$$

用相交面积乘该 bin 的电场，再对覆盖区域求和。

大单元往往得到更大的原始密度力。

因此后面还需要预条件，统一不同单元的移动尺度。



一个 cell 覆盖多个不同的电场采样点

原理示意 · 非实测运行

$\Sigma \, a_i b \, E_\beta$

面积加权

合成密度力

> Scatter → Poisson → Gather，是密度计算最值得记住的数据流。

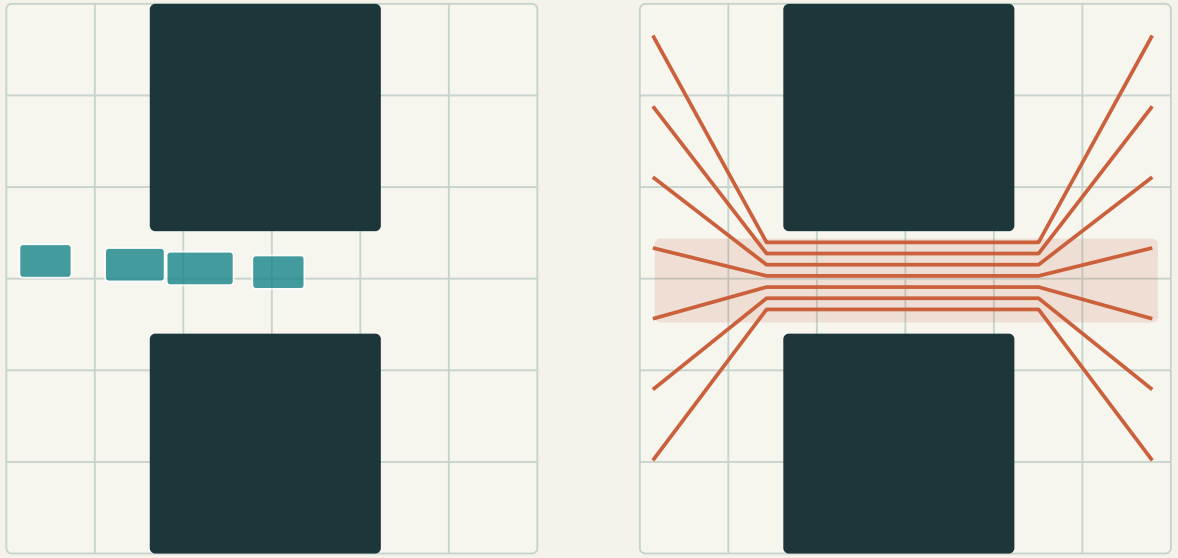
来源：密度力回采样 ↗ · 面积散射与 overflow ↗

密度低，不代表导线能穿过去

密度图：实例模型面积 / 可放置容量。

拥塞图：连线需求 / 可用 routing 资源。

两块宏之间的狭窄通道，可能几乎没有 cell，却承载大量跨区连接。



密度占用　　布线需求

同一通道，两个指标 · 几何与连线均为教学示意

> 判断可布线性需要连线与容量信息；只看单元面积不够。

来源：面积散射与 overflow ↗ · RouteBase · 拥塞反馈 ↗

预条件：把“力大”与“移动过猛”分开

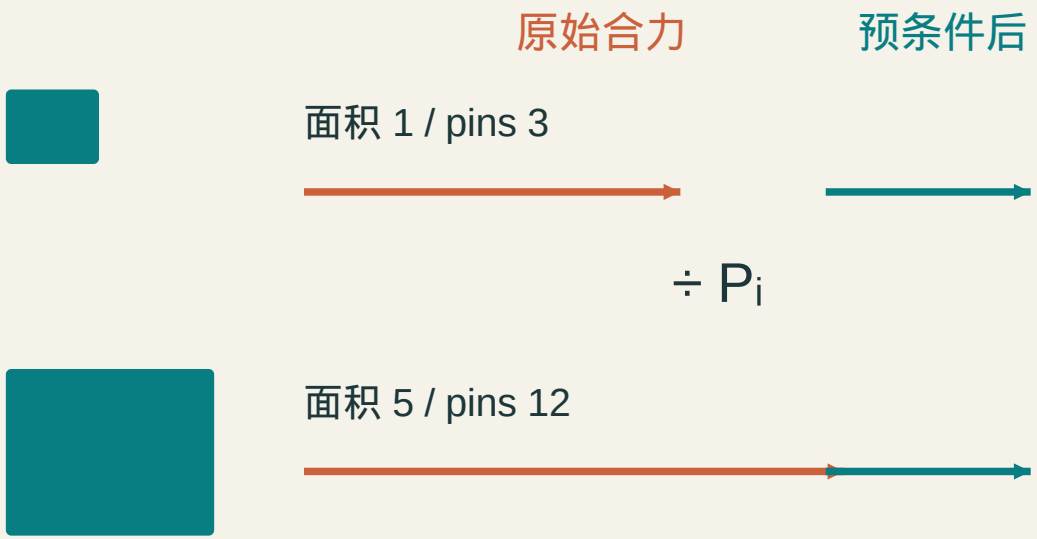
$$d_i = (F_{W,i} + \lambda F_{D,i}) / P_i$$

$$P_i \approx \max(P_{min}, n_{pin,i} + \lambda A_i)$$

F_W 与 F_D 已经指向下降方向。

pin 多、面积大的单元，分母更大。

这是对角尺度修正，不是精确求 Hessian 逆矩阵。



原理示意 · 非实测运行

> 代码按 z ← z + αd 更新；别把“gradient”变量再取一次负号。

来源：力合成与预条件 ↗ · WA 导数与符号 ↗

λ 的起点：先匹配两种力的量级

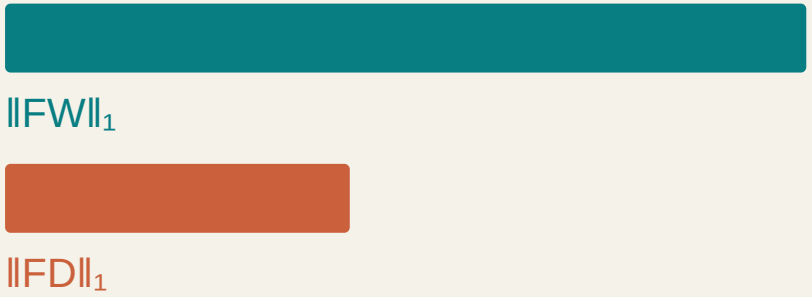
$$\lambda_0 = c_0 \cdot \|F_W\|_1 / \|F_D\|_1$$

先计算全体相关 cell 的原始力绝对值之和。

c_0 控制初始的相对密度强度。

当密度力总量为零时，源码使用回退值，避免直接除零。

原始力的 L1 总量



按比例设定 λ_0

先匹配量级，再用 c_0 调整初始偏好

原理示意 · 非实测运行

λ 的绝对数值依赖尺度；离开坐标单位、面积和力，单看数字没有意义。

来源：密度罚系数初始化 ↗ · 力合成与预条件 ↗

λ 随迭代变化，把拥挤布局逐步展开

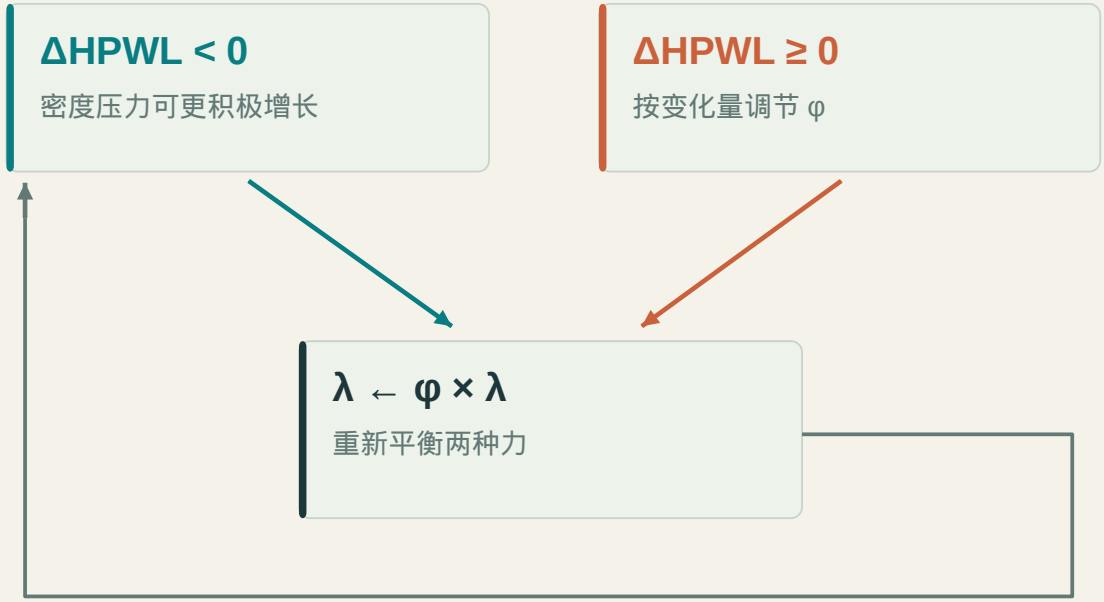
$$\lambda_{k+1} = \varphi_k \lambda_k$$

$$\Delta = (HPWL_k - HPWL_{k-1})/H_{ref}$$

HPWL 改善时，允许更积极增加密度压力。

HPWL 变差时，增长因子按 Δ 调整，并设下界。

实际 φ 还受动态调节与恢复逻辑影响。



原理示意 · 非实测运行

> 线长允许适度增长，以换取较低的空间冲突。

来源：罚系数迭代调度 ↗ · Nesterov 主循环 ↗

Nesterov：在外推位置计算下一步

$$u_{k+1} = y_k + \alpha_k d(y_k)$$

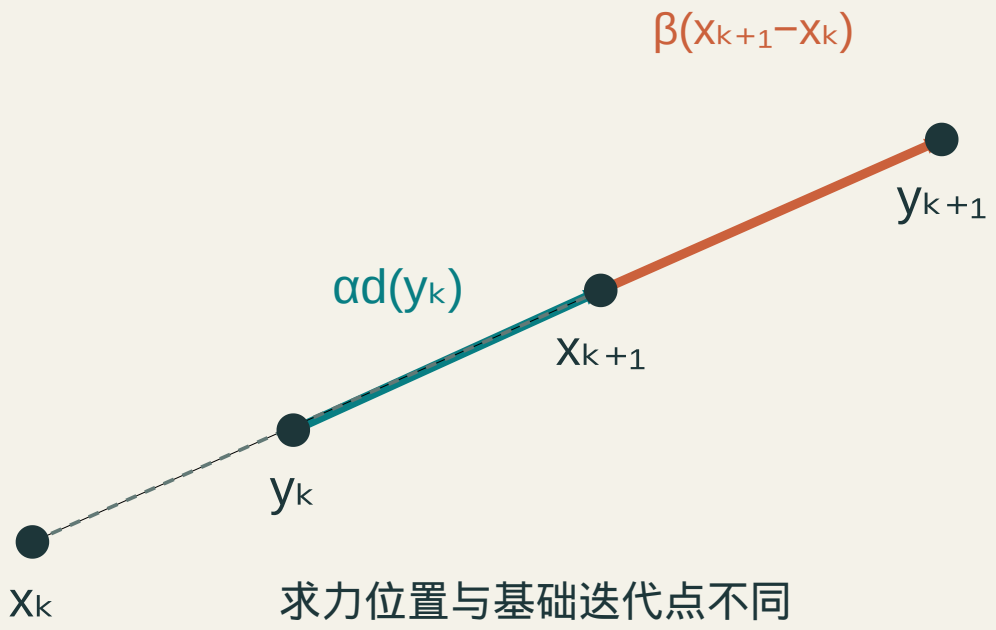
$$x_{k+1} = \Pi\Omega(u_{k+1})$$

$$y_{k+1} = \Pi\Omega[u_{k+1} + \beta_k(u_{k+1} - x_k)]$$

x：基础点；y：外推点；u：未裁剪试探点。

$a_{k+1} = (1 + \sqrt{(1+4a_k^2)})/2$

$\beta_k = (a_k - 1)/a_{k+1}$；$\Pi\Omega$ 表示边界处理。



求力位置与基础迭代点不同

原理示意 · 非实测运行

> 沿当前方向移动，再利用历史位移外推；边界处理限制可行范围。

来源：Nesterov 坐标更新 ↗ · Nesterov 主循环 ↗

读代码时，分清 Cur / SLP / Next 的身份

数学角色	主要源码字段
x_k	curCoordi_
y_k	curSLPCoordi_
$d(y_k)$	curSLPSumGrads_
x_{k+1} / y_{k+1}	nextCoordi_ / nextSLPCoordi_

试探点计算新场和新力后，接受迭代，再轮换缓冲区。



> 几何、pin 位置、密度场和梯度，必须对应同一轮坐标。

来源：Nesterov 坐标更新 ↗ · 力合成与预条件 ↗ · 罚系数迭代调度 ↗

实验 ③：加速从何而来，又为何会越过最优点

$$f(x,y) = \tfrac{1}{2}(x^2 + 12y^2)$$

同一起点 (8,4)、同一步长 α = 1/12。

比较普通梯度下降与 Nesterov；每次点击执行一轮。

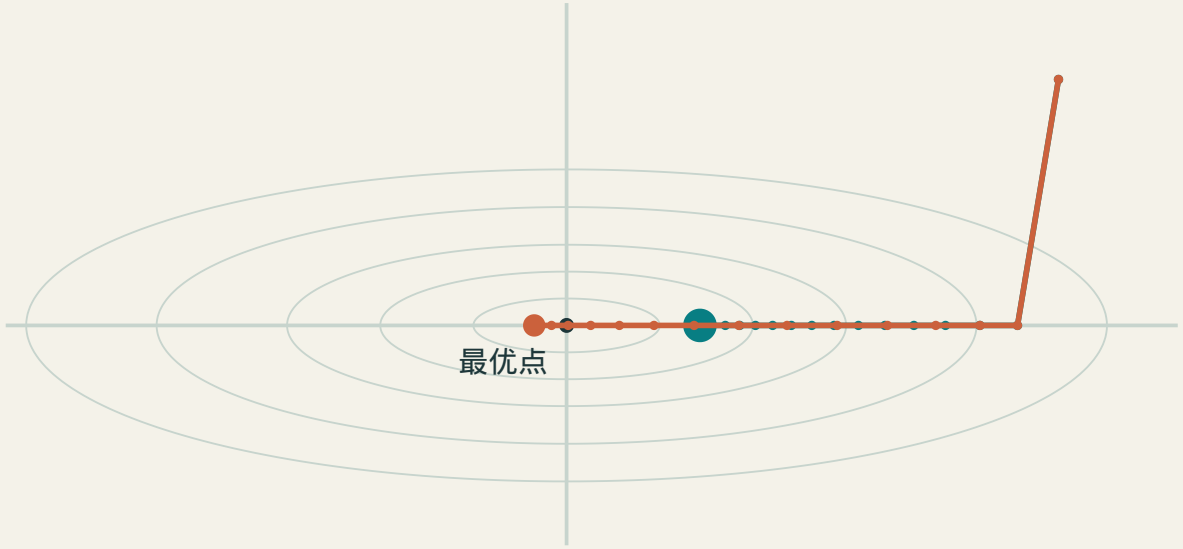
迭代 15 / 40

f(GD) = 2.352302

f(Nesterov) = 0.138479

凸二次示例仅演示动量，不包含 GPL 的密度、预条件与反馈。

f = ½(x² + 12y²)



青：GD　　橙：Nesterov　　k = 15

教学数值实验 · 非完整 GPL 运行

> 加速常能更快穿过平缓方向，但它不是每步单调下降的保证。

来源：Nesterov 坐标更新 ↗ · Nesterov 主循环 ↗

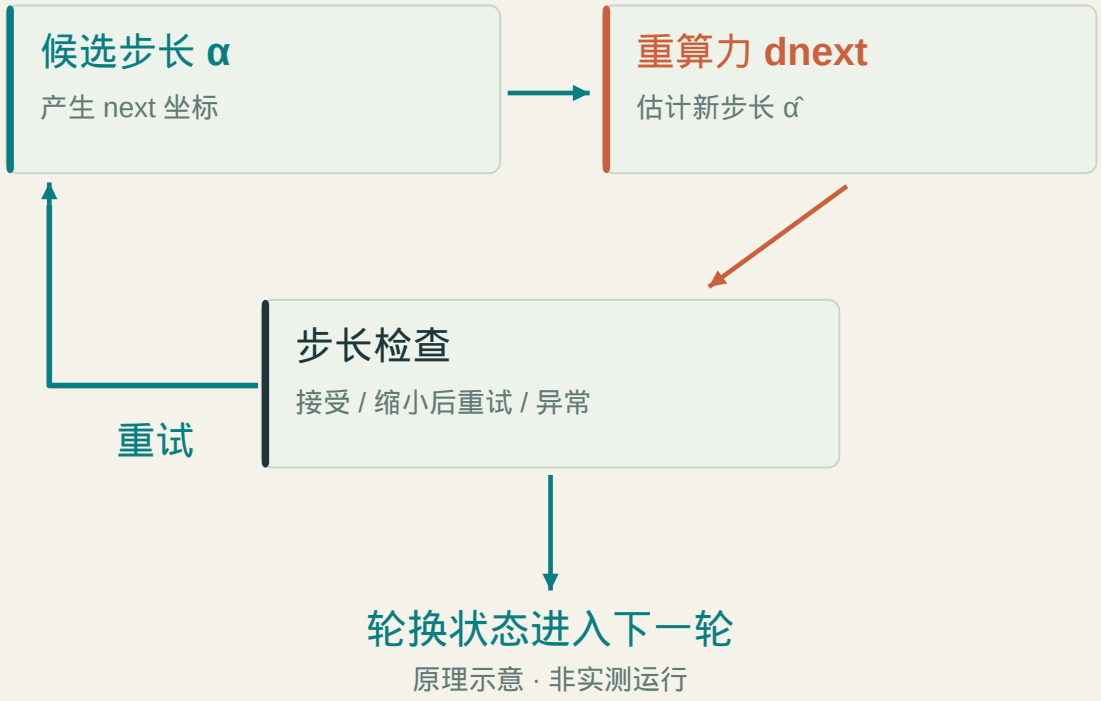
步长来自局部变化，而不是一次设定后不再改变

$$\hat{\alpha} \approx \|y_{new} - y_{old}\|_2 \,/\, \|d_{new} - d_{old}\|_2$$

试探新坐标 → 重算场和力 → 比较新旧步长。

力变化很大：局部更陡，需要较小步长。

零分母、NaN、Inf 和试探上限都需要保护。

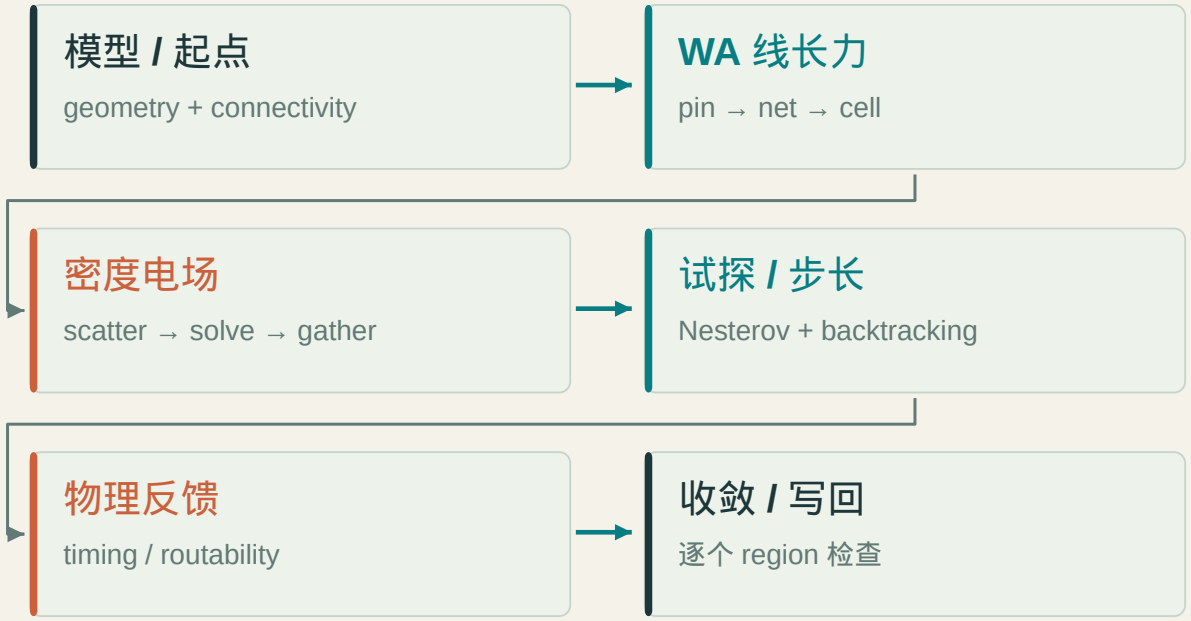


> 这里不是基于目标值的标准 Armijo line search。

来源：步长估计与检查 ↗ · Backtracking ↗

把一轮循环展开：昂贵工作发生在哪里

1. 计算动量系数，生成试探坐标。
2. 更新 pin / bin，完成 WA 与 Poisson。
3. 合成力，估计步长；必要时重试。
4. 接受状态，更新 overflow、λ 与平滑系数。
5. 触发时序 / 拥塞反馈，检查收敛与异常。



未收敛：带着更新后的模型回到求力步骤

原理示意 · 非实测运行

> 优化运行时间时，先统计每轮算了几次场、多少次反馈与重启。

来源：Nesterov 主循环 ↗ · Backtracking ↗

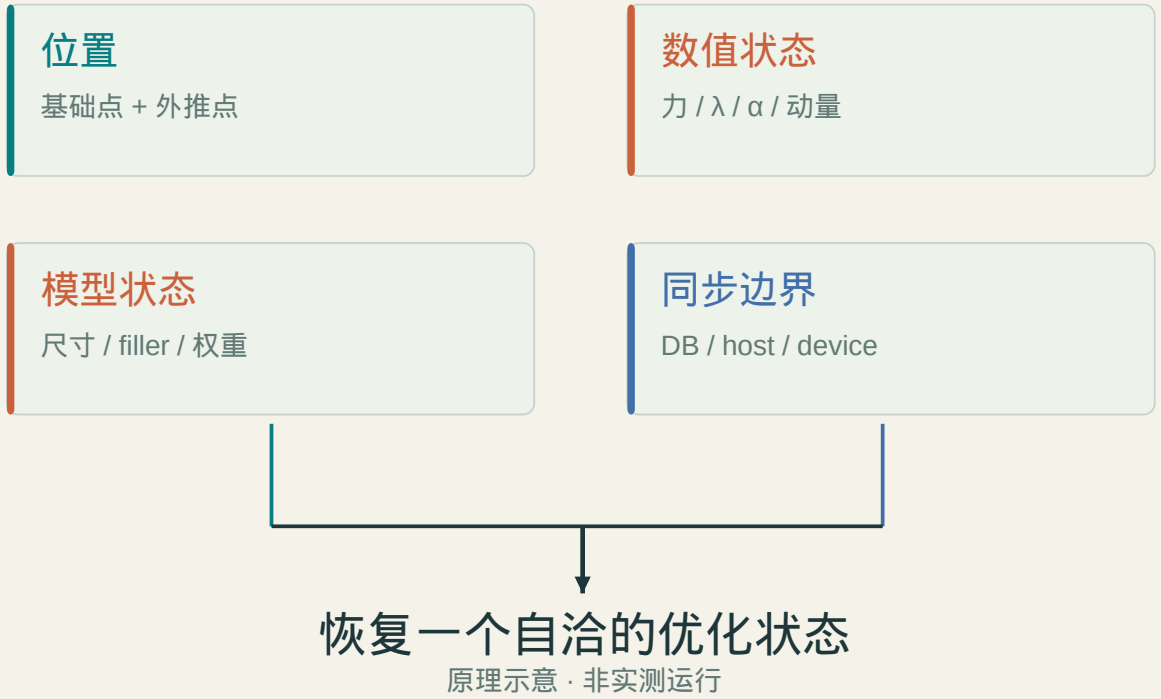
快照与回退：保存的是一致状态，而非一张坐标表

快照可能涉及坐标、梯度、罚系数、动量与线长平滑系数。

低 overflow 附近出现退化时，检查已保存的较好解。

拥塞反馈还需要管理虚拟尺寸、filler 与目标密度。

NaN / Inf 是异常信号；达到迭代上限也不等于成功收敛。



> 恢复后如果状态来自不同轮次，再好的坐标也无法保证下一步正确。

来源：发散快照与恢复 ↗ · 拥塞重启与快照 ↗

真实回归曲线：展开使 HPWL 上升，overflow 下降

0.8655 → 0.0990

密度 overflow

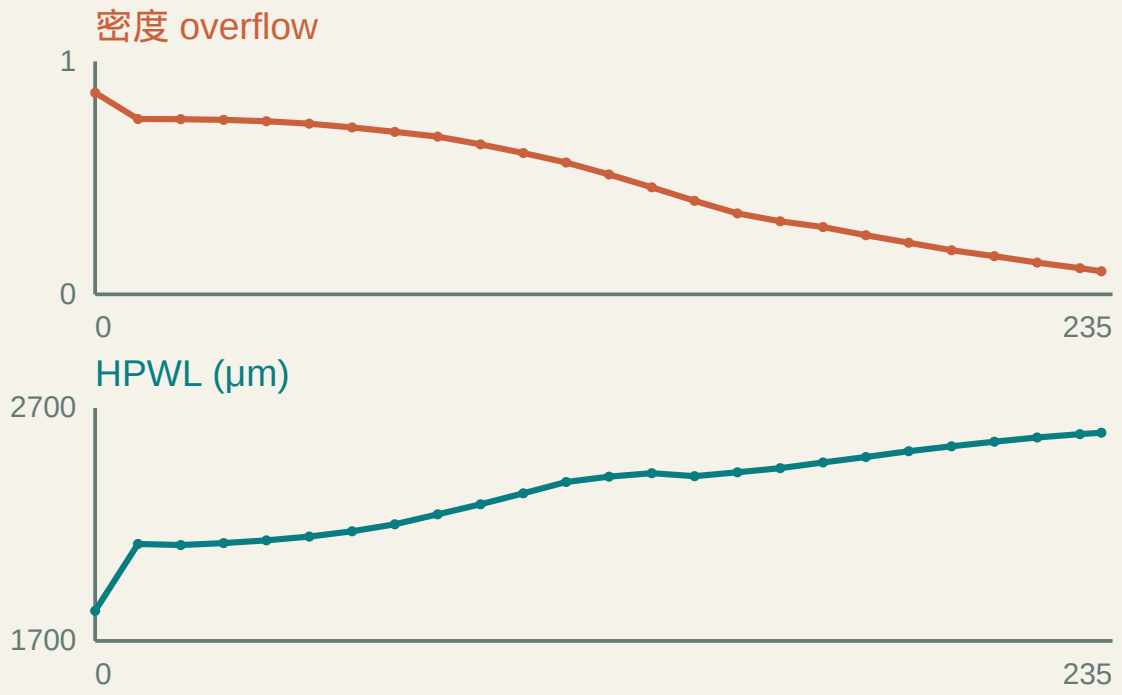
1829.304 → 2594.019 µm

迭代日志中的 HPWL

235 次迭代

本例目标 overflow = 0.1

最终 DB 报告 HPWL = 2597.56 µm，口径不同；未在本次重新运行。



来自固定提交 simple01.ok · 25 个日志采样点 · 横轴为迭代

> 收敛判据主要看各 region 的 overflow；仍需检查后续合法化与布线质量。

来源：simple01.ok · 已有 golden ↗ · 区域收敛判定 ↗

时序驱动：让关键连接更有“拉力”

位置 → 估计寄生 → STA / resizer → net slack。

选出较关键的 nets，增加它们在线长目标中的权重。

后续优化缩短关键连接，再评估新的关键路径。

反馈在 overflow 阶段条件下触发，不是每轮完整跑 STA。



原理示意 · 非实测运行

> 关键路径是动态的：布局改变延迟，延迟又改变布局优先级。

来源：TimingBase · net 重加权 ↗ · 时序反馈编排 ↗

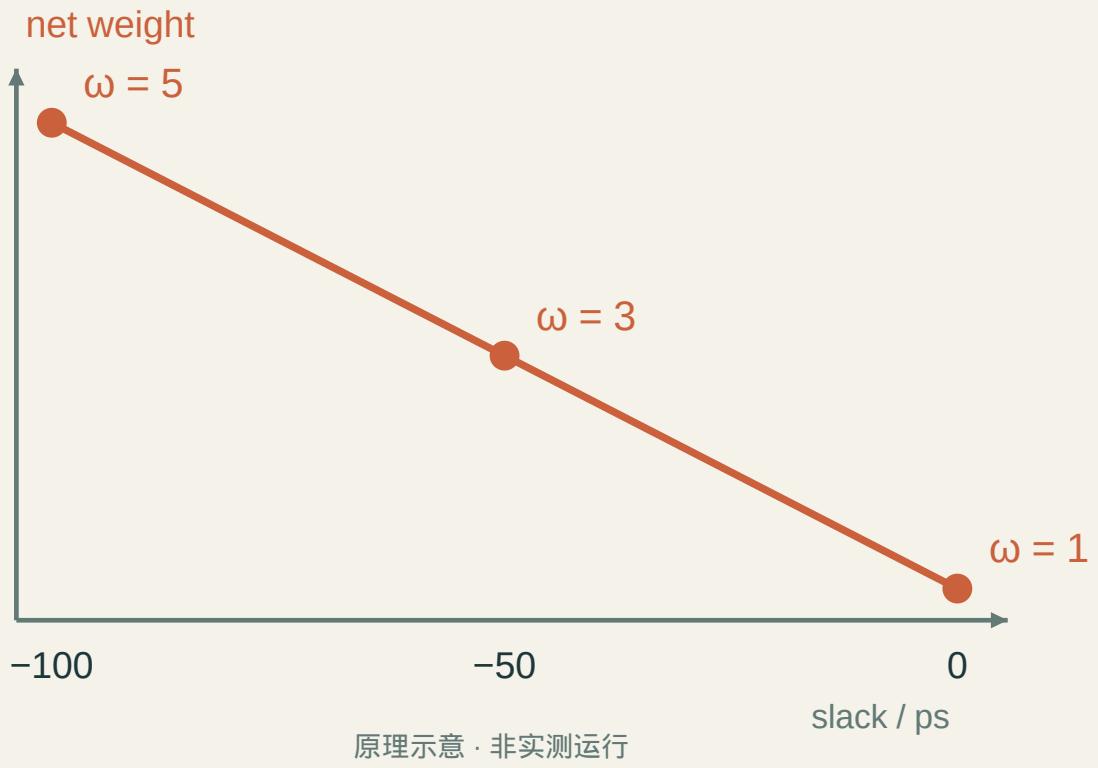
从 slack 到 net 权重：线性映射的可解释例子

$$\omega(s) = 1 + (\omega_{max} - 1) \cdot (s_{max} - s)/(s_{max} - s_{min})$$

例：$s_{min} = -100$ ps，$s_{max} = 0$，$\omega_{max} = 5$。

slack −100 / −50 / 0 ps

对应权重 **5 / 3 / 1**。



原理示意 · 非实测运行

> 未获得有效 slack 或未进入加权范围的 net 保持基础权重。

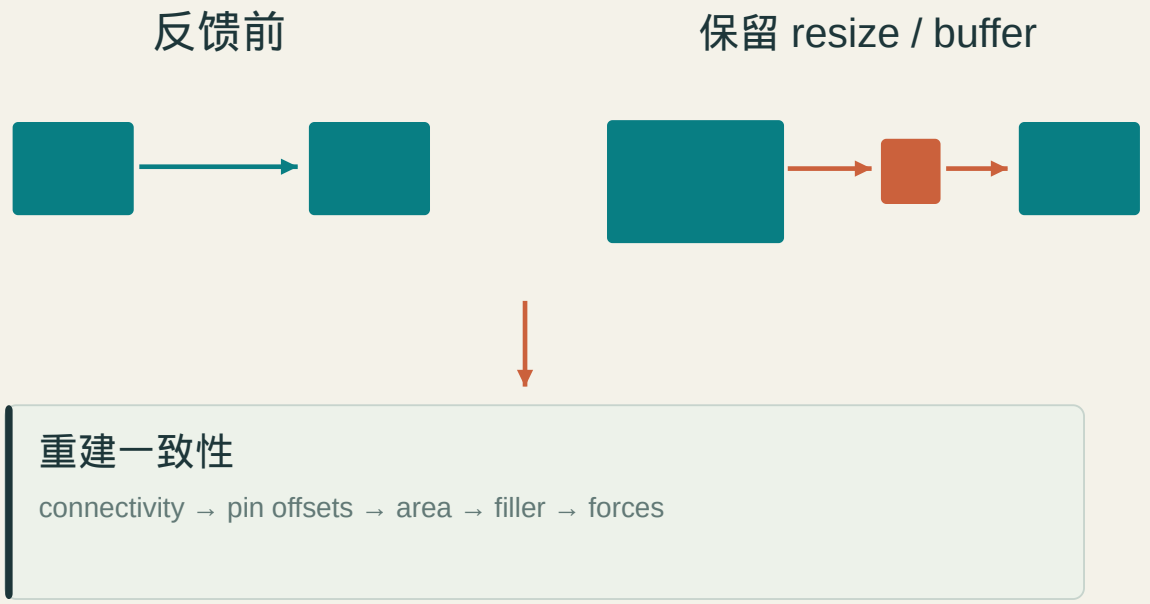
来源：TimingBase · net 重加权 ↗

Resizer 不只提供分数，也可能改变优化对象

换 cell master → 面积和 pin offset 改变。

插入 / 删除实例与 net → 索引和连接关系改变。

虚拟分析可回滚修改；保留修改时，GPL 必须同步更新内部对象与面积预算。



原理示意 · 非实测运行

> 一次 timing feedback 后，旧的指针、梯度与 filler 预算可能都已失效。

来源：TimingBase · net 重加权 ↗ · 时序反馈编排 ↗ · GCell / GPin / GNet ↗

时序结果的可信度，取决于评估模型

早期位置估计不了所有绕线、拥塞、层选择与 via。

实际 CTS 尚未完成，时钟延迟模型会改变关键路径。

当前分支的 Virtual CTS 可临时注入时钟插入延迟，结束前移除。

最终比较必须跟踪到同一后续阶段的 WNS / TNS。



原理示意 · 非实测运行

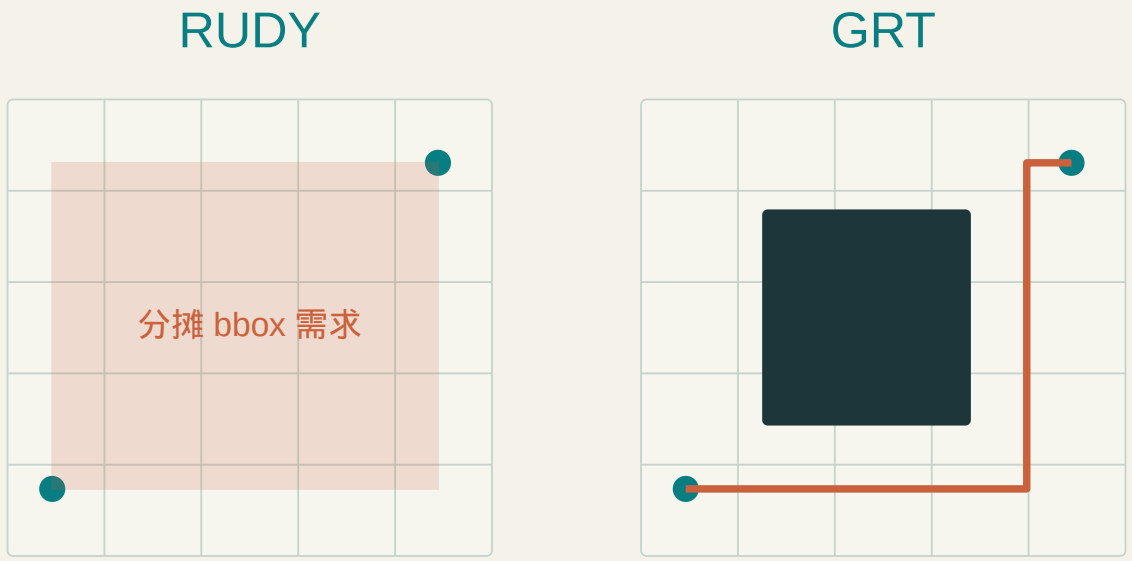
> 改善 placement slack 是中间信号；布线后结果才验证代理是否有效。

来源：Virtual CTS ↗ · 时序反馈编排 ↗ · GPL 模块说明 ↗

RUDY 快速估需求，GRT 提供更具体的路由反馈

RUDY	GRT
按 net 包围盒分摊需求	使用全局布线资源与结果
适合快速反复评估	包含更具体的绕路与容量信息
几何代理，有盲区	成本较高，仍非详细布线

当前 GPL 默认走 RUDY，可选择 GRT。



快速几何代理 带资源与绕路信息

原理示意 · 非实测运行

> 反馈越真实通常越昂贵；两种估计都需要后续详细布线验证。

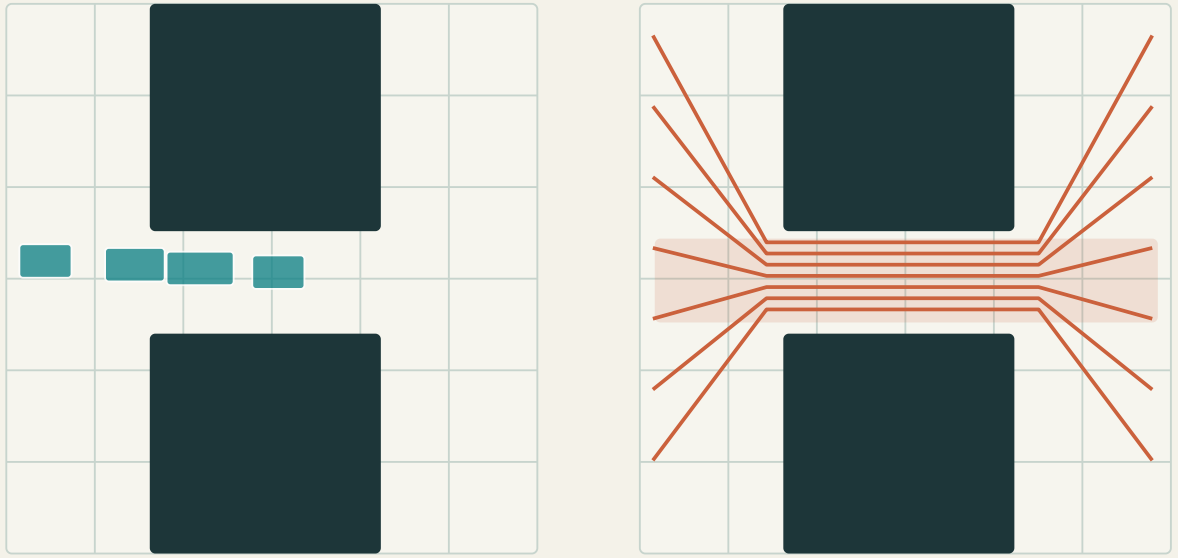
来源：RouteBase · 拥塞反馈 ↗ · GPL 模块说明 ↗

宏块通道：短线长方案可能把问题集中到一条缝

紧凑聚类使大量跨组连接争用中间通道。

适当分散单元，可让连线从更多位置通过。

若通道容量先天不足，仍需修改 floorplan 或宏块位置。



同一通道，两个指标 · 几何与连线均为教学示意

> 算法可以重新分配需求，却无法凭空创造 routing capacity。

来源：RouteBase · 拥塞反馈 ↗ · Tile 膨胀比例 ↗

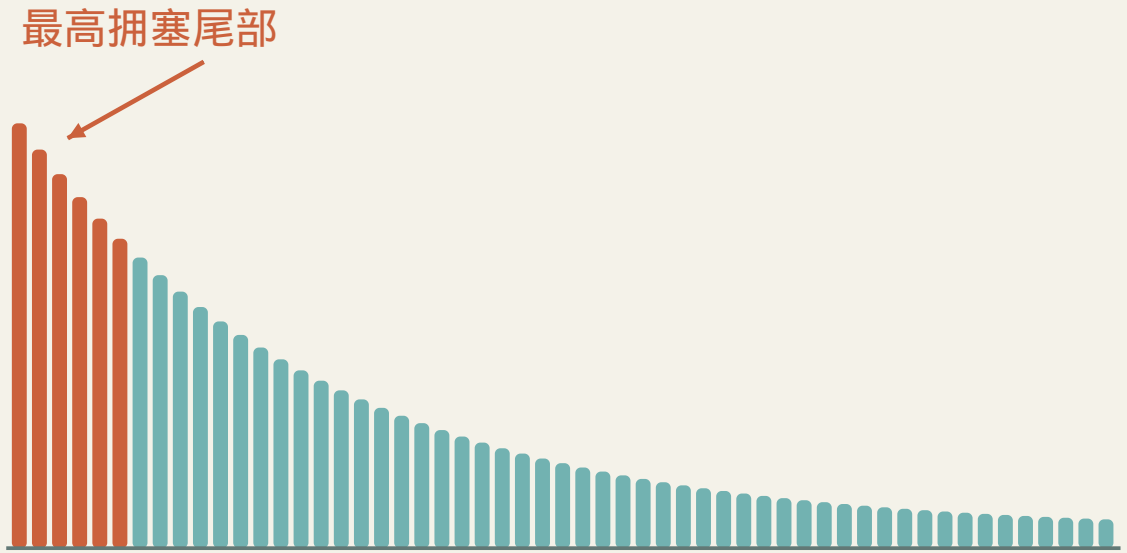
RC metric 关注最拥塞的尾部，而非全芯片平均

把拥塞值排序，分别取最高 **0.5%**、**1%**、**2%**、**5%** 的平均。

$$RC = \Sigma\, k_j \cdot RC_{top\text{-}j} \,/\, \Sigma\, k_j$$

GRT 路径还比较水平 / 垂直方向的尾部值。

此处 RC 指 routing congestion，和电学 R、C 参数不是同一概念。



从高到低排序 → 取多个尾部区间均值

原理示意 · 非实测运行

> 评价指标决定优化重点；全局均值可能掩盖少量致命热点。

来源：拥塞尾部统计 ↗ · RouteBase · 拥塞反馈 ↗

膨胀：让拥塞区域的 cell 在模型里“更占地方”

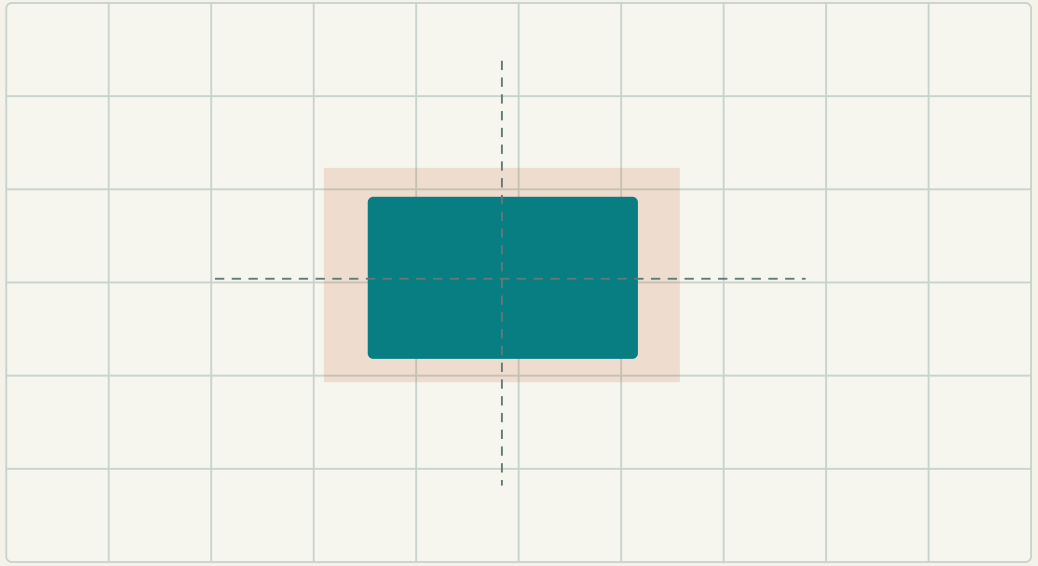
$$r = \min(r_{max}, (c/c_0)^p)\ ，\ c \geq c_0$$

$$w' \approx w\sqrt{r}\ ，\ h' \approx h\sqrt{r}$$

c：tile 拥塞；c_0：膨胀阈值；r：面积倍率。

真实 master 尺寸不因这种膨胀而改变。

新增虚拟面积先消耗 filler / 空间预算。



真实 cell → 较大的优化模型

实心：真实尺寸 · 橙框：虚拟尺寸 · 中心不因膨胀自动移动

> 拥塞膨胀增加模型面积；密度平滑只是重新分布同一面积。

来源：Tile 膨胀比例 ↗ · 单元膨胀与 filler 预算 ↗

实验 ④：拥塞比例如何转化为面积压力

$c_0 = 0.95$，$p = 2$，$r_{max} = 3$。

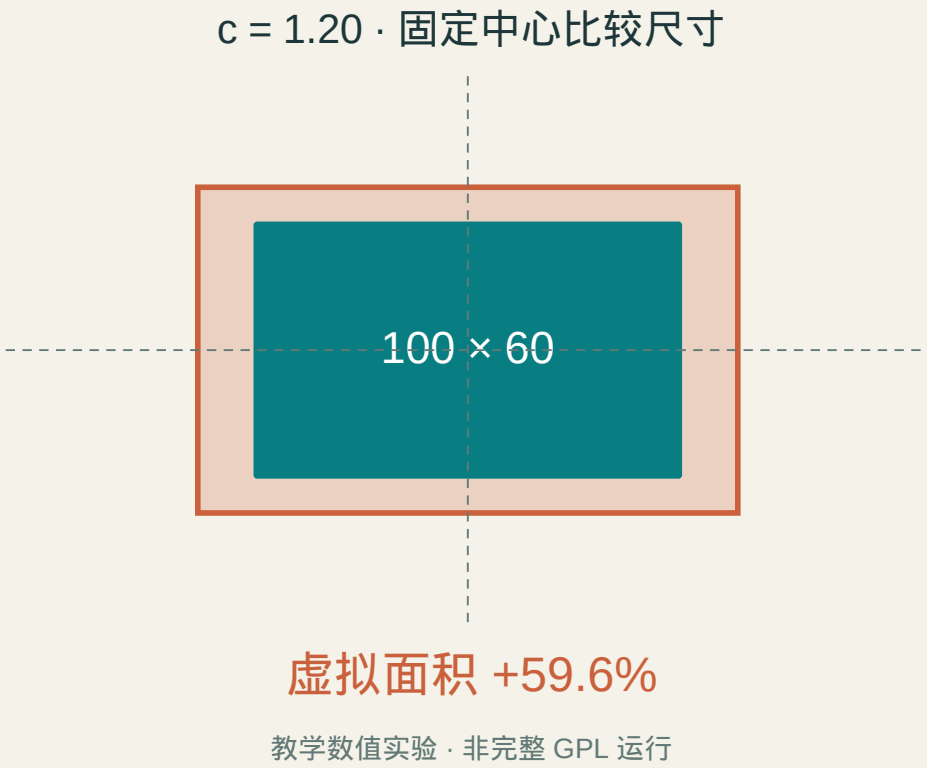
教学 cell 尺寸：100 × 60 DBU。

理论面积倍率 r = 1.5956

模型尺寸 = 126 × 76 DBU

舍入后实际面积倍率 = 1.5960

只计算一次膨胀；单元中心保持不变，不模拟后续重新布局。



> 面积倍率为 r，边长倍率为 √r；不要把两者都乘 r。

来源：Tile 膨胀比例 ↗ · 单元膨胀与 filler 预算 ↗

膨胀之后必须重新展开，必要时恢复较好的状态

达到合适阶段 → 估拥塞 → 更新最佳记录。

若仍需优化：膨胀、调整 filler / 密度，恢复布局快照继续 Nesterov。

达到目标、预算受限或连续不改善时，结束或回退。

当前分支还检查移动是否稳定，避免在高速迁移时采集失真的拥塞反馈。



原理示意 · 非实测运行

> 位置快照与最佳拥塞的尺寸 / 权重记录，是两套相关但不同的状态。

来源：RouteBase · 拥塞反馈 ↗ · 拥塞重启与快照 ↗

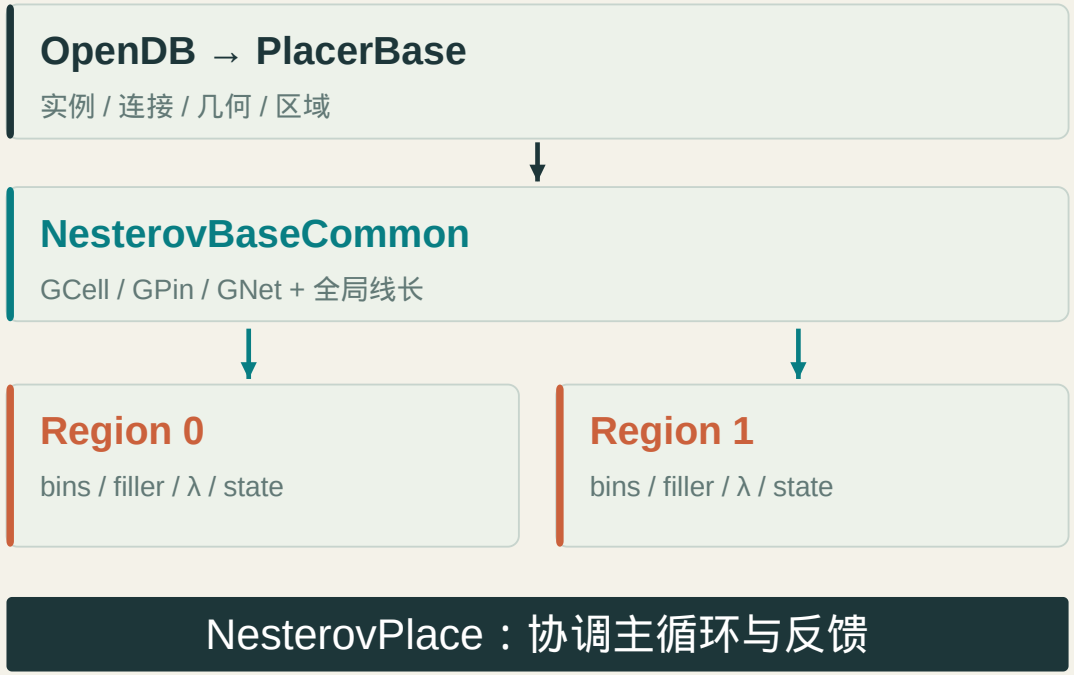
源码只需要先认清四个核心责任

PlacerBase：从 OpenDB 提取几何、连接和区域。

NesterovBaseCommon：全局 GCell / GPin / GNet 与线长计算。

NesterovBase：每个 region 的 bins、filler、密度与迭代状态。

NesterovPlace：协调迭代、时序、拥塞、收敛和写回。



原理示意 · 非实测运行

> 按“谁拥有数据、谁改变状态、谁发起重算”读代码。

来源：PlacerBase · 数据建模 ↗ · GCell / GPin / GNet ↗ · Nesterov 主循环 ↗

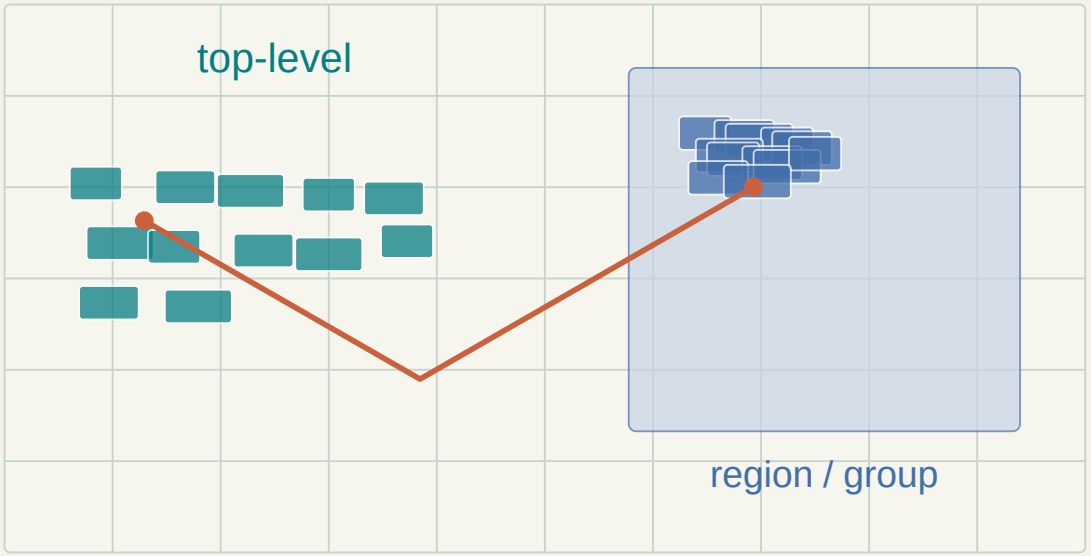
多 region：线长全局耦合，密度在各自区域求解

跨区 net 仍连接不同区域的 pins，线长力需要共同计算。

每个 region 有自己的 bins、filler、λ 和 overflow。

主循环协调全部 region；全部满足条件才能正常结束。

宏块、边界与非矩形可用空间需要正确建模。



线长跨域耦合；密度分区管理

原理示意 · 非实测运行

> 某个小 region 的容量错误，可能影响整个设计的收敛。

来源：PlacerBase · 数据建模 ↗ · Nesterov 主循环 ↗ · 区域收敛判定 ↗

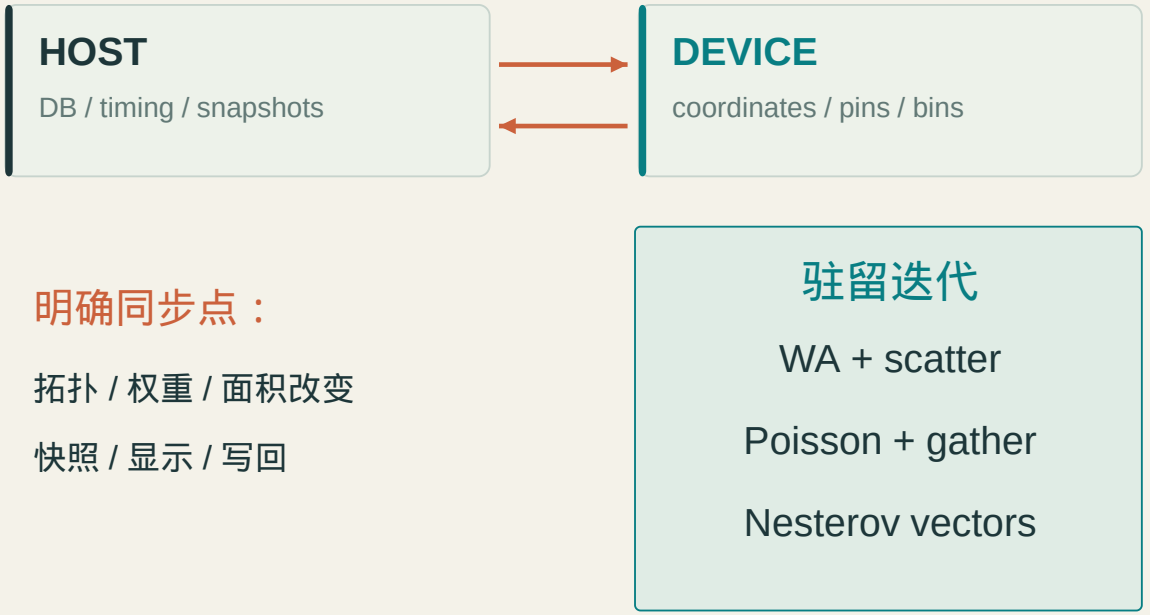
CPU / GPU：数学算子相同，数据新鲜度成为重点

热点：WA / HPWL、面积 scatter、FFT、密度 gather、向量更新。

设备驻留减少逐轮 host/device 往返。

时序修改、可视化、快照和最终写回需要明确同步边界。

验证质量、数值容差与可重复性；不能只看 kernel 时间。



原理示意 · 非实测运行

> 这份演示不提供 GPU 加速比：没有本次端到端测量证据。

来源：GPU 迭代上下文 ↗ · 力合成与预条件 ↗ · CPU DCT Poisson 求解 ↗

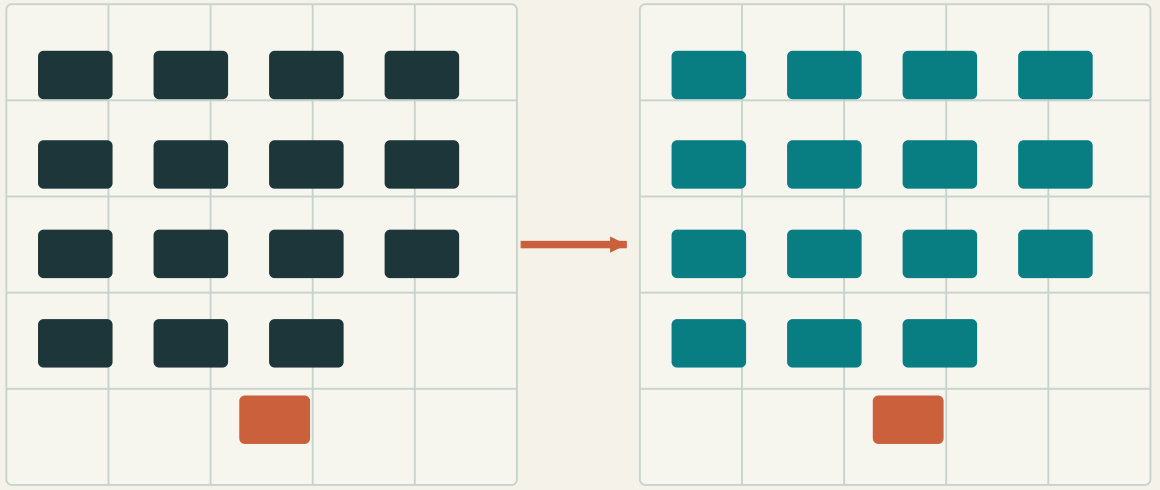
增量布局：先处理新增对象，再决定是否继续松动

当前两阶段路径：先锁住已有 PLACED 实例，放置未放置对象。

阶段一后解除临时锁；若发散或未达到目标，则继续第二阶段。

第二阶段可加强密度罚项；永久 fixed 对象仍按固定语义处理。

全已放置等情形有不同分支，不能套用同一流程。



阶段 1：临时锁　　必要时阶段 2：继续优化

原理示意 · 非实测运行

> “增量”描述重用已有布局的策略，不承诺所有旧单元零位移。

来源：增量布局两阶段 ↗

读日志：先定位机制，再调整参数

观察	优先检查
overflow 长期不降	可用面积、region、锁、密度力与 λ
HPWL 突然跳变	TD 修改、RD 重启、快照、权重
步长 **NaN / Inf**	零力差、非法面积 / 坐标、尺度
低密度但 **routing** 失败	宏通道、pin access、资源与拥塞



原理示意 · 非实测运行

> 把异常追到产生错误状态的那一步，再看输出和日志。

来源：Nesterov 主循环 ↗ · 发散快照与恢复 ↗ · 迭代 overflow 分母 ↗

评价一次改动：让早期收益走到后续流程

固定输入网表、floorplan、库 / RC / SDC、seed、线程与版本。

每次只改变一个因素：普通 GPL → TD → RD → 组合。

记录 GPL 曲线与时间，也跟踪 DPL 位移、GRT 拥塞、DRT 违例和 routed WNS / TNS。

01 **GPL**
HPWL / overflow

02 **DPL**
位移 / 合法性

03 **GRT**
资源 / 拥塞

04 **DRT**
违例 / WNS / TNS

原理示意 · 非实测运行

> 更低 HPWL、更快收敛、更好 routed timing，是三个需要分别验证的结论。

来源：GPL 模块说明 ↗ · GPL 回归测试 ↗

三个练习，让新人从“听懂”走向“能定位”

1. 读曲线：从 simple01.ok 找一次 HPWL 上升与 overflow 下降，解释为什么合理。
2. 跟踪一次力：从一个 GPin 的 WA 导数追到 GCell 更新，确认符号和预条件。
3. 跟踪一次膨胀：记录尺寸、filler、overflow 和快照在 RD 前后的变化。

有构建环境时，从 src/gpl/test 的小用例开始；测试输出放入独立结果目录。

一条 net

pin → WA → cell force

一个 bin

overlap → density → field

一次更新

force → α / β → coordinates

原理示意 · 非实测运行

> 每个练习都应有可检查的答案或数据，而不只是“成功跑完”。

来源：GPL 回归测试 ↗ · simple01.ok · 已有 golden ↗

源码阅读顺序：每次只回答一个问题

问题	入口
起点如何得到？	initialPlace.cpp → solver.cpp
线长力来自哪里？	updateWireLengthForceWA_native
空间力来自哪里？	BinGrid → FFT → getDensityGradient
位置如何更新？	doNesterovPlace → nesterovUpdateCoordinates
物理反馈如何介入？	TimingBase / RouteBase



原理示意 · 非实测运行

> 先建立公式与状态的对应关系，再读异常路径和后端优化。

来源：InitialPlace · B2B 建模 ↗ · WA 线长与力 ↗ · 面积散射与 overflow ↗ · CPU DCT Poisson 求解 ↗ · Nesterov 主循环 ↗ · TimingBase · net 重加权 ↗ · RouteBase · 拥塞反馈 ↗

用五个问题检查是否真的理解了 GPL

1. 为什么 HPWL 上升时，布局可能正在改善？
2. 为何 filler 参与密度，却不计入 overflow 分子？
3. 密度平滑和拥塞膨胀分别守恒什么？
4. 源码为什么用“加梯度”更新坐标？
5. 为什么低 overflow 不能保证布线成功？



原理示意 · 非实测运行

> 能用“公式 + 图 + 源码位置”解释这些问题，就已经掌握主要框架。

来源：WA 导数与符号 ↗ · 密度尺寸与缩放 ↗ · 面积散射与 overflow ↗ · Tile 膨胀比例 ↗

把几组容易混淆的名词放在一起

记号 / 概念	含义
γ / wlCoeff	线长平滑尺度 / 其倒数形式
λ / densityPenalty	密度项的相对强度
ψ / phiCoef	电势 / λ 更新倍率，两者不同
α / β	步长 / Nesterov 动量倍率
bin / routing tile	放置密度网格 / 布线评估网格
scaled / unscaled	实现中的面积与 overflow 路径，不等于有 / 无目标密度



原理示意 · 非实测运行

> overflow 是比例，HPWL 是长度，area 是面积；比较前先核对单位与归一化。

来源：面积散射与 overflow ↗ · 迭代 overflow 分母 ↗ · 平滑系数调度 ↗ · 罚系数迭代调度 ↗

源码基线、证据和延伸阅读

实现基线

zint99/OpenROAD

`977813f3b02e35692b7b5ab3cca36e4e6fa380bb`

算法背景

Lu et al. · ePlace · TODAES 2015

Cheng et al. · RePlAce · TCAD 2019

实数证据

simple01.ok / simple01.defok / nangate45.lef

交互例子是教学公式；未提供新的 OpenROAD 性能测量。

固定提交

源码 / tests / diagrams 的共同坐标

原理与实数证据分开标识

教学公式 ≠ GPL 仿真；golden ≠ 本次测量

所有资源可离线阅读

SVG / HTML / PDF / 逐页讲解稿

原理示意 · 非实测运行

> 论文解释思想，固定版本源码解释当前行为；两者不能逐项画等号。

来源：ePlace · Lu et al., TODAES 2015 ↗ · RePlAce · Cheng et al., TCAD 2019 ↗ · simple01.ok · 已有 golden ↗ · simple01.defok · 已有布局 ↗

从连接图到位置场，再回到真实芯片

连接告诉我们靠近谁。

密度告诉我们空间在哪里。

物理反馈告诉我们哪些位置更有价值。

Nesterov 把这些信息变成一轮轮可计算的位移。

下一步：选一条 net、一个 cell、一个 bin，追完一整轮数据流。



> 理解 GPL 的核心，是理解每次移动背后的信息、尺度和状态。

来源：Nesterov 主循环 ↗ · 力合成与预条件 ↗ · TimingBase · net 重加权 ↗ · RouteBase · 拥塞反馈 ↗